

Cooling Tower Optimisation

Task 6 (Group Report)



**Pierre BEDRUNE, Martin HARRAMBAT, Clémence-Philomène HINOT,
John HOARAU**

Prepared for March 6, 2026

Abstract

This report addresses the structural optimisation of a natural-draught cooling tower, modelled as a vertical stack of $m = 10$ conical frustums. The objective is to minimise the total lateral surface area, a proxy for construction material cost, subject to a fixed internal volume constraint of $V_{\text{target}} = 70,320 \text{ m}^3$. The equality constraint is handled via a quadratic penalty method, enabling a uniform, solver-agnostic problem formulation.

Three optimisation algorithms are benchmarked: the quasi-Newton Broyden-Fletcher-Goldfarb-Shanno (BFGS) method, Particle Swarm Optimisation (PSO), and Simulated Annealing (SA). Their performance is evaluated across eight study cases of increasing complexity, ranging from a fixed-height reference case ($D = 9$) to a parametric hyperboloidal fit ($D = 3$), encompassing variations in dimensionality, initialisation strategy, constraint topology, and physical scaling.

Results demonstrate that NEED TO GET THE FINAL RESULTS

Acronyms

BFGS Broyden-Fletcher-Goldfarb-Shanno

PSO Particle Swarm Optimisation

SA Simulated Annealing

Contents

1	Introduction	7
1.1	Project Context and Progression	7
1.2	Problem Statement	8
1.3	Report Structure	8
1.4	Statement on the Use of Generative AI	9
2	Cooling Tower	10
2.1	Mathematical Problem Formulation and Geometric Parametrisation	10
2.2	Methodology: Quadratic Penalty Formulation and Search Land- scape	11
2.3	Architectural Decoupling and Benchmark Fairness	12
2.4	Study Cases: Dimensionality and Physical Complexity	12
3	Algorithms	14
3.1	Particle Swarm Optimisation method	14
3.1.1	General explanation & Algorithm	14
3.1.2	Seed exploration	16
3.2	Simulated Annealing method	18
3.2.1	Principle	18
3.2.2	Acceptance Criterion (Metropolis)	19
3.2.3	Cooling Schedule	19
3.2.4	Key Parameters	19
3.2.5	Multi-Objective Strategy (Weight Scalarisation)	19
3.3	Broyden-Fletcher-Goldfarb-Shanno method	20

3.3.1	Principle	20
3.3.2	Mathematical Formulation	20
3.3.3	Key Parameters	21
3.3.4	Constraint Handling	22
4	Results	23
4.1	Particle Swarm Optimisation results	23
4.1.1	Convergence	23
4.1.2	Profiles	26
4.2	Simulated Annealing results	30
4.2.1	Case 1: Mandatory Reference Case	30
4.2.2	Case 2: Variable Heights	31
4.2.3	Case 3: Full Freedom	32
4.2.4	Case 4: Tight Waist	33
4.2.5	Case 5: Cylindrical Start	34
4.2.6	Case 6: High Volume	35
4.2.7	Case 7: Restricted Bounds	36
4.2.8	Case 8: Hyperbolic Fit	37
4.3	Broyden-Fletcher-Goldfarb-Shanno results	38
4.3.1	Case 1: Mandatory Reference Case	38
4.3.2	Case 2: Variable Heights	39
4.3.3	Case 3: Full Freedom	40
4.3.4	Case 4: Tight Waist	41
4.3.5	Case 5: Cylindrical Start	42
4.3.6	Case 6: High Volume	43

4.3.7	Case 7: Restricted Bounds	44
4.3.8	Case 8: Hyperbolic Fit	45
4.4	Comparison between the three methods	46
4.5	Comparison of the methods performances	46
5	Discussion	47
5.1	Particle Swarm Optimisation	47
5.1.1	Convergence analysis	47
5.1.2	Profile analysis	48
5.1.3	Best result	48
5.2	SA	51
5.3	BFGS	51
5.4	Global discussion	51
6	Conclusion	52
6.1	Conclusion on Simulated Annealing	52
	References	54
7	Appendices	55
	Appendix A Code Listings	55
	Appendix A.A Problem Definition — <code>cooling_tower.py</code>	55
	Appendix A.B Solver Orchestrator — <code>solver.py</code>	62
	Appendix A.C BFGS Implementation — <code>bfgs.py</code>	81
	Appendix A.D PSO Implementation — <code>pso.py</code>	87
	Appendix A.E SA Implementation — <code>sa.py</code>	92
	Appendix B Detailed Results Data	95

Appendix B.A BFGS Results	95
Appendix B.B PSO Results	95
Appendix B.C SA Results	95

1 Introduction

Structural optimisation is a cornerstone of modern computational engineering, enabling the design of lighter, more efficient, and more cost-effective structures. Among the most widely studied civil structures in this context is the natural-draught cooling tower, whose hyperboloidal shell geometry presents a rich landscape of continuous, non-linear optimisation challenges [1]. Minimising the lateral surface area of such a tower, directly proportional to construction material consumption, while satisfying a strict volumetric capacity constraint is a problem of both theoretical interest and direct industrial relevance.

This report constitutes Task 6 of the Digital Engineering Design Group Project undertaken by Group Chenonceau within the MSc in Computational and Software Techniques in Engineering at Cranfield University. The group comprises John Hoarau (Project Lead and Integration), Martin Harambat (BFGS Implementation), Pierre Bédrupe (Simulated Annealing), and Clémence-Philomène Hinot (Particle Swarm Optimisation and L^AT_EX).

1.1 Project Context and Progression

Over the preceding tasks, we as a group progressively built our optimisation competence through a structured sequence of increasingly complex problems. In Task 3, two metaheuristic algorithms, Particle Swarm Optimisation (PSO) and Simulated Annealing (SA), were independently developed and validated against two canonical benchmarks: the discrete Gear Train Design problem and the multi-objective Two-Bar Plane Truss Design problem. Task 4 then introduced a deterministic gradient-based counterpart, the Broyden-Fletcher-Goldfarb-Shanno (BFGS) quasi-Newton method, applied to the same benchmarks to enable direct algorithmic comparison. Key outcomes included the charac-

terisation of BFGS superlinear convergence properties, the evaluation of noise robustness across all three methods, and the identification of the complementary strengths and weaknesses of stochastic versus deterministic approaches.

Task 6 represents the culmination of this progression. For the first time, all three algorithms are deployed together on a single, real-world structural optimisation problem: the minimisation of the lateral surface area of a discretised hyperboloidal cooling tower subject to a fixed volume constraint.

1.2 Problem Statement

The cooling tower is modelled as a vertical stack of m conical frustums, yielding a constrained nonlinear programme (NLP) in which the decision variables, subsets of the internal radii and section heights, are optimised to minimise the total lateral surface area while preserving a target internal volume $V_{\text{target}} = 70,320 \text{ m}^3$. The equality constraint is handled via a quadratic penalty method, enabling uniform treatment across all three solvers without introducing algorithm-specific constraint logic.

To comprehensively evaluate algorithmic performance, eight study cases of increasing complexity are defined. These range from a fixed-height reference case ($D = 9$ decision variables) to a parametric hyperboloidal fit ($D = 3$), encompassing variations in decision variable type, dimensionality, initialisation strategy, constraint topology, and physical scaling.

1.3 Report Structure

The remainder of this report is organised as follows. Section 2 presents the mathematical formulation of the cooling tower problem, the quadratic penalty methodology, the software architecture, and the rationale behind the eight study cases. Section 3 de-

tails the three optimisation algorithms and their respective adaptations to this problem. Section 4 reports the results across all cases and algorithms. Section 5 provides a comparative discussion on convergence behaviour, constraint satisfaction, and computational cost. Finally, Section 6 draws conclusions and identifies directions for future work.

1.4 Statement on the Use of Generative AI

During the preparation of this work, the authors used ChatGPT (OpenAI) and Claude AI (Anthropic) to assist with code efficiency optimisation and debugging, technical writing refinement, and literature review formatting. After using these tools, all content was reviewed and edited as needed. The authors take full responsibility for the final report's content, analysis, and code functionality.

2 Cooling Tower

2.1 Mathematical Problem Formulation and Geometric Parametrisation

The cooling tower of a nuclear power plant defines a continuous surface optimisation problem over a rotationally symmetric structure. To render this tractable computationally, the continuous domain is discretised into a vertical stack of m conical frustums, bounded by $m + 1$ horizontal cross-sections (rings). Let z denote the vertical axis originating at the tower base, with $z_0 = 0$. Each frustum $i \in \{1, \dots, m\}$ is fully characterised by its bottom radius r_{i-1} , top radius r_i , and section height h_i .

The macroscopic engineering objective is to minimise construction material costs, which are directly proportional to the total lateral surface area A_{total} . This is subject to a strict equality constraint enforcing a fixed thermodynamic cooling capacity, expressed as a target volume $V_{\text{target}} = 70,320 \text{ m}^3$. For each frustum i , the slant height s_i , lateral surface area A_i , and volumetric capacity V_i are given by:

$$s_i = \sqrt{(r_{i-1} - r_i)^2 + h_i^2} \quad (1)$$

$$A_i = \pi(r_{i-1} + r_i) s_i \quad (2)$$

$$V_i = \frac{\pi h_i}{3} (r_{i-1}^2 + r_{i-1}r_i + r_i^2) \quad (3)$$

The resulting global problem is formally a constrained nonlinear programme (NLP):

$$\min_{\mathbf{x} \in \mathcal{D}} A_{\text{total}}(\mathbf{x}) = \sum_{i=1}^m A_i(\mathbf{x}) \quad \text{subject to} \quad \mathcal{H}(\mathbf{x}) = \sum_{i=1}^m V_i(\mathbf{x}) - V_{\text{target}} = 0 \quad (4)$$

where $\mathbf{x}$ is the decision variable vector, and $\mathcal{D} \subseteq \mathbb{R}^D$ is the feasible space defined by

the physical boundary conditions.

2.2 Methodology: Quadratic Penalty Formulation and Search Landscape

To evaluate the three fundamentally different algorithms, Simulated Annealing (SA), Particle Swarm Optimisation (PSO), and the quasi-Newton BFGS method, on a common, unbiased basis, the equality constraint $\mathcal{H}(\mathbf{x}) = 0$ must be handled uniformly. Since SA and PSO are natively unconstrained, and since introducing solver-specific constraint-handling logic would compromise benchmark fairness, the equality constraint is absorbed into the objective via a **quadratic penalty method**.

The surrogate objective becomes:

$$J(\mathbf{x}) = A_{\text{total}}(\mathbf{x}) + \rho [\mathcal{H}(\mathbf{x})]^2 \quad (5)$$

where $\rho \gg 0$ is a static scalar penalty weight. A static ρ is deliberately chosen over an adaptive schedule to ensure that all three algorithms operate on an identical, fixed search landscape any dynamic ρ strategy would introduce algorithm-dependent trajectory effects that would confound the performance comparison. The chosen value $\rho = 10^3$ was calibrated empirically to ensure that volume violations incur a penalty several orders of magnitude larger than the surface area objective, effectively rendering infeasible solutions non-competitive without distorting the local geometry near the feasible manifold.

This penalty formulation has two notable consequences on the search landscape. First, for BFGS, the quadratic term degrades the conditioning of the Hessian approximation in proportion to ρ , increasing the curvature of the surrogate near constraint boundaries [2]. Second, for SA and PSO, the penalty creates steep regions of high curvature in the search landscape around infeasible points, placing greater demands on the exploration-

exploitation balance of each algorithm.

2.3 Architectural Decoupling and Benchmark Fairness

A fully decoupled, object-oriented software architecture was implemented around a central `CoolingTowerProblem` class, which encapsulates the geometric model and all case-specific logic. The key design principle is a vector mapping pipeline (`unpack_decision_variables`): all three solvers interact exclusively with an abstract decision vector $\mathbf{x} \in \mathbb{R}^D$, with no knowledge of the underlying physical structure. The class internally reconstructs the full radii and heights arrays from $\mathbf{x}$ according to the active case. This architecture ensures that the `solver.py` orchestrator applies each algorithm identically across all eight cases, with no solver-specific preprocessing or constraint logic injected at the call level. The result is a genuinely fair, one-to-one algorithmic comparison.

2.4 Study Cases: Dimensionality and Physical Complexity

Eight cases were designed to progressively stress-test the algorithms across increasing levels of search space complexity, physical realism, and parametric dimensionality. All cases are evaluated against BFGS, SA, and PSO.

- **Case 1: Mandatory Reference Case ($D = 9$):** The canonical benchmark. Section heights h_i and boundary radii $r_0 = 39.3$ m, $r_m = 27.4$ m are fixed, and the nine inner radii $r_1, \dots, r_9$ constitute the sole decision variables. This low-dimensional, well-conditioned problem serves as the convergence baseline against which all other cases are evaluated.
- **Case 2: Variable Heights ($D = 10$):** The radii profile from Case 1 is fixed, and the ten section heights h_i are optimised instead. This isolates the vertical

sensitivity of the volume constraint and characterises how each algorithm navigates a height-dominated feasible manifold.

- **Case 3: Full Freedom ($D = 19$):** Both inner radii and section heights are optimised simultaneously. The near-doubling of dimensionality substantially expands the search landscape, exposing each algorithm to a higher density of local minima and testing resilience against premature convergence.
- **Case 4: Tight Waist ($D = 9$):** A structural constraint is imposed: $r_{\text{mid}} < 0.7 \min(r_0, r_m)$, artificially narrowing the tower throat. This breaks the convexity of the feasible region and introduces a disconnected, non-convex feasible zone, testing each solver's ability to navigate segmented constraint boundaries.
- **Case 5: Cylindrical Initialisation ($D = 9$):** A deliberately poor starting point is imposed: $r_i = r_0$ for all i , corresponding to a cylinder that substantially violates the volume constraint. This case measures the sensitivity of each algorithm to initialisation, and in particular the speed at which each method escapes a far-from-feasible equilibrium.
- **Case 6: High Volume ($D = 9$):** The target volume is increased by 50% to $V_{\text{target}} = 105,480 \text{ m}^3$. This rescales the feasible manifold and alters the relative magnitude of the penalty gradient $\nabla[\mathcal{H}(\mathbf{x})]^2$, probing each algorithm's scalability under a shifted constraint landscape.
- **Case 7: Restricted Bounds ($D = 9$):** Box constraints are imposed on all inner radii: $r_i \in [20, 60] \text{ m}$, simulating civil engineering construction tolerances. This tests BFGS boundary approximations via L-BFGS-B, and requires SA and PSO to implement explicit boundary clipping or reflection strategies.

- **Case 8: Hyperbolic Parametrisation ($D = 3$):** Rather than optimising discrete node positions, the tower profile is constrained to a hyperboloid of one sheet:

$$r(z) = a\sqrt{1 + \frac{(z - c)^2}{b^2}} \quad (6)$$

where a is the waist radius, b controls the curvature, and c is the height of the waist. The optimisation operates directly on (a, b, c) , reducing the search to $D = 3$. This parametrisation substantially mitigates the curse of dimensionality while inherently guaranteeing C^1 profile continuity—a structurally significant property, as smooth surface curvature improves wind load distribution along the tower shell [1]. This case also enables a direct comparison between the discrete frustum approximation and the analytically integrated surface area of the true hyperboloid, quantifying the geometric discretisation error introduced in Cases 1–7.

3 Algorithms

3.1 Particle Swarm Optimisation method

[3]

3.1.1 General explanation & Algorithm

The Particle Swarm Optimisation (PSO) is a stochastic optimisation technique inspired by the social behaviour of birds. A population of candidate solutions moves through the domain. Through several time steps, each particle adjusts its position based on its personal best and on the global best. This allows the algorithm to converge to a solution that matches a criterion [4].

As the problem is single-objective, the standard PSO is used to minimise one error

function. It uses two major concepts:

- Continuous Movement: Particles move in continuous space according to standard velocity and position update rules.
- Convergence Tracking: The best error is stored at each iteration to generate a convergence curve.

The PSO algorithm initialises a population of particles distributed across the search space, each assigned a random velocity and position. At each iteration, velocities are updated according to each particle's own experience and the collective knowledge of the swarm, weighted by an inertia term that decreases progressively to balance exploration and exploitation. A perturbation of the global best position is introduced to reduce the risk of converging prematurely to a local optimum. The algorithm stops when either the maximum number of iterations is reached or the improvement in the global best falls below a defined tolerance, ensuring computational efficiency without sacrificing solution quality.

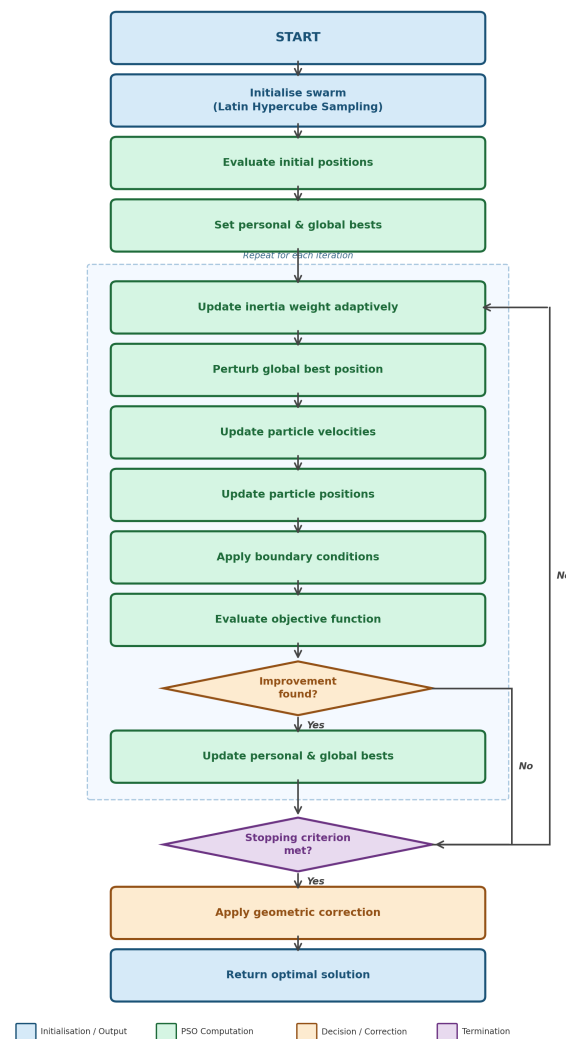


Figure 1: Overview of the Particle Swarm Optimisation algorithm

3.1.2 Seed exploration

To assess the influence of stochastic initialisation on the stability of the PSO optimisation, a seed-sensitivity study was conducted. Seven distinct seeds were tested including a random one called "None". The resulting volume errors were evaluated across the eight test cases. The objective of this exploration was to select a seed that provides the best results.

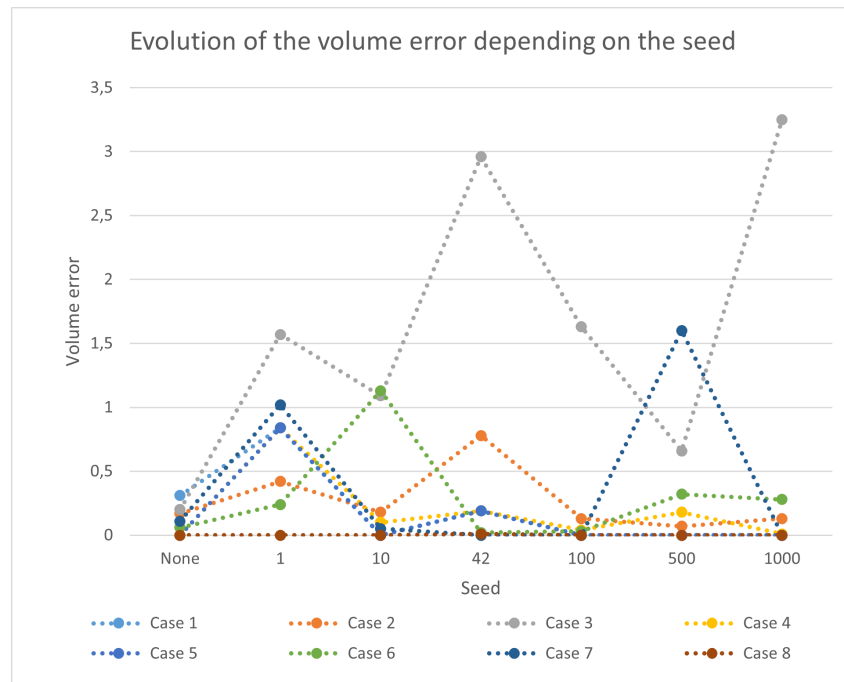


Figure 2: Evolution of the volume error depending on the seed

Overall, the results show marked variability depending on the seed. In particular, some seeds lead to pronounced peaks for specific cases. The most notable example is the seed of 1000, which generates the largest error on the case 3 in the entire exploration. These large deviations indicate that these seeds may place the optimiser in initial conditions less favourable for convergence or stability.

In contrast, seeds 1 and 10 tend to produce substantially lower and more homogeneous errors across the eight cases. The seed 10 performs particularly well: it shows no extreme outliers, and the volume error remains close to zero for most cases. Importantly, its behaviour is stable - neither systematically over-optimistic nor overly sensitive to case-to-case variations. Compared to seed 1, seed 10 shows a slightly smoother profile, with fewer local spikes.

The seed option labelled "None", corresponding to the use of a non-deterministic

initialisation, was deliberately not retained. Although its average behaviour appears reasonable, its intrinsic randomness means that results cannot be reproduced reliably accross runs. In the context of this study - chere traceability, repeatability, and controlled comparisons are essential - this lack of determinism is undesirables. For this reason, the random seed configuration was excluded from the potential candidates.

Based on this analysis, the seed 10 was selected for the remainder of the study. The choice is justified by its balances behaviour:

- consistently low volume errors accross all cases,
- absence of irregular peaks,
- good reproducibility.

This seed therefore offers a reliable compromose between robustness and computational stability. However, it is important to note that the fluctuations are minors compared to the total volume of the cooling tower.

3.2 Simulated Annealing method

3.2.1 Principle

Simulated Annealing is an optimisation method inspired by the annealing process in metallurgy [5]. The algorithm explores the search space by accepting neighbouring solutions according to a probabilistic criterion: improvements are always accepted, while degradations are accepted with a probability that decreases over time (decreasing temperature). This acceptance mechanism helps escape local minima at the beginning, then allows the algorithm to converge towards a global minimum [5].

3.2.2 Acceptance Criterion (Metropolis)

The probability of accepting a worse solution is given by:

$$P(\text{acceptance}) = e^{-\frac{E_{\text{new}} - E_{\text{actual}}}{T}} \quad (7)$$

where E_{actual} is the current energy, E_{new} is the energy of the new solution, and T is the temperature parameter [5].

3.2.3 Cooling Schedule

At each iteration, the temperature decreases according to:

$$T = T \times \alpha \quad (8)$$

where α is the cooling rate [5].

3.2.4 Key Parameters

The SA algorithm requires the following parameters:

- T_{initial} : Initial temperature (controls exploration intensity)
- α (**cooling_rate**): Cooling speed (typically 0.95–0.999)
- T_{min} : Stopping temperature

3.2.5 Multi-Objective Strategy (Weight Scalarisation)

SA is inherently a mono-objective algorithm. To solve bi-objective problems, a scalarisation approach transforms the problem into a single objective function using a weighted sum:

$$E(x) = w_1 \cdot f_1(x) + w_2 \cdot f_2(x) + \sum_i \lambda_i \cdot \max(0, g_i(x)) \quad (9)$$

where $w_1 + w_2 = 1$ and λ_i are penalty coefficients for constraint violations. To approximate a Pareto front, multiple independent runs are performed with weights w_1 uniformly distributed between 0.005 and 0.995 [5].

3.3 Broyden-Fletcher-Goldfarb-Shanno method

3.3.1 Principle

The BFGS (Broyden-Fletcher-Goldfarb-Shanno) algorithm is a quasi-Newton method designed to accelerate the iteration process in nonlinear optimisation [6]. Unlike Newton's method, which requires explicit computation of the Hessian matrix, BFGS builds an approximation of the Hessian using only gradient information collected during the optimisation process.

The method generates a sequence of iterates x_k starting from an initial guess x_0 . At each iteration, it computes a search direction p_k using the current Hessian approximation B_k and gradient ∇f_k , then performs a line search to determine the step length α_k . The algorithm terminates when convergence criteria are satisfied, typically when the gradient norm falls below a specified tolerance [6].

3.3.2 Mathematical Formulation

The BFGS method approximates the inverse Hessian $H_k \approx (\nabla^2 f_k)^{-1}$ and updates it at each iteration using the following formula [6]:

$$H_{k+1} = (I - \rho_k s_k y_k^T) H_k (I - \rho_k y_k s_k^T) + \rho_k s_k s_k^T \quad (10)$$

where:

$$s_k = x_{k+1} - x_k \quad (\text{step taken}) \quad (11)$$

$$y_k = \nabla f_{k+1} - \nabla f_k \quad (\text{change in gradient}) \quad (12)$$

$$\rho_k = \frac{1}{y_k^T s_k} \quad (13)$$

The search direction is computed as:

$$p_k = -H_k \nabla f_k \quad (14)$$

The new iterate is obtained via line search:

$$x_{k+1} = x_k + \alpha_k p_k \quad (15)$$

where α_k satisfies the Wolfe conditions to ensure sufficient decrease and curvature conditions.

3.3.3 Key Parameters

The BFGS algorithm requires the following parameters:

- x_0 : Initial guess
- **Gradient tolerance**: Convergence threshold for $\|\nabla f_k\|$
- **Step tolerance**: Minimum step size before termination
- **Maximum iterations**: Upper bound on iteration count

3.3.4 Constraint Handling

For constrained problems, a penalty approach transforms the objective function:

$$F(x) = f(x) + \lambda \sum_i \max(0, g_i(x))^2 \quad (16)$$

where $g_i(x) \leq 0$ are the constraint functions and λ is the penalty coefficient.

4 Results

4.1 Particle Swarm Optimisation results

4.1.1 Convergence

To better characterise the optimisation dynamics, the convergence histories are analysed in pairs. All curves exhibit the typical behaviour of PSO, with a rapid initial decay followed by a gradual stabilisation once the swarm approaches a locally optimal region. However, the rate of decrease, the number of iterations required to reach the plateau, and the presence of intermittent stagnation phases vary significantly from one case to another.

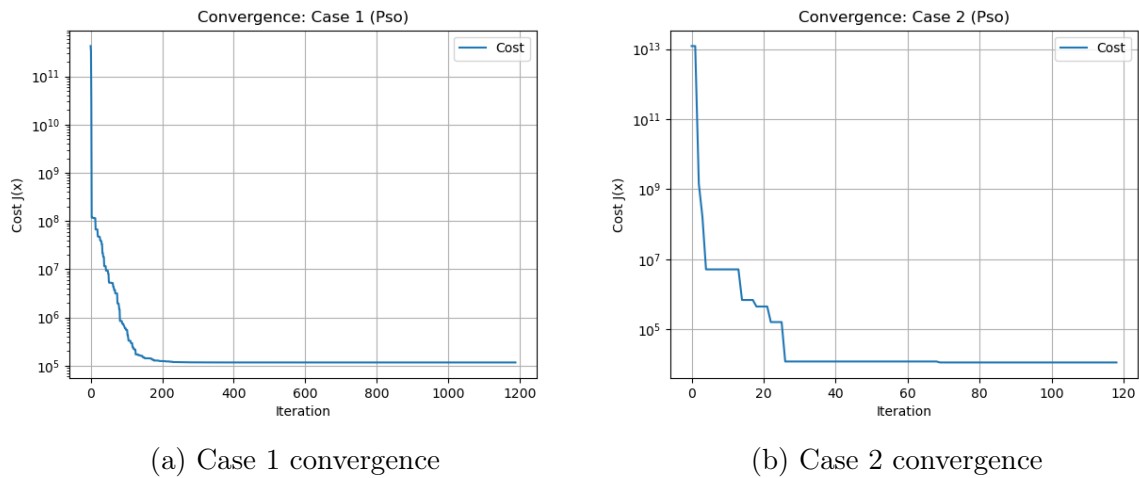
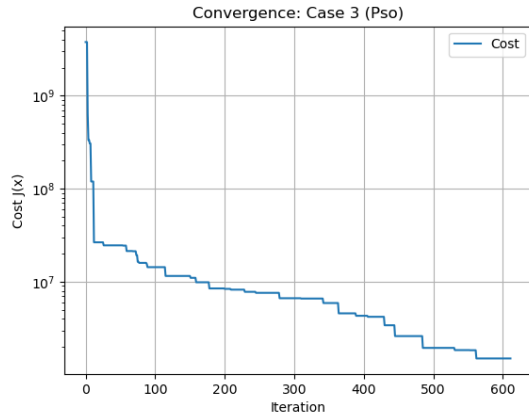


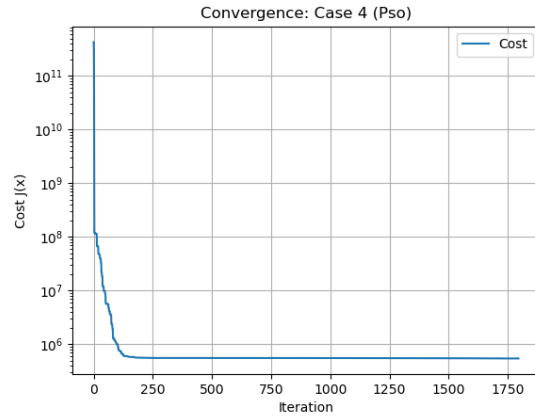
Figure 3: Convergence plots (cases 1 and 2)

Both cases 1 and 2 show a steep cost reduction during the first iterations, dropping several orders of magnitude within the first 20 to 40 iterations. In case 1, the convergence then continues more progressively until 150 iterations, before entering a stable regime where the cost remains essentially constant for the remainder of the run. Case 2 reaches its stabilisation region sooner, around 30 iterations, after which the curve remains flat

with only minor adjustments.



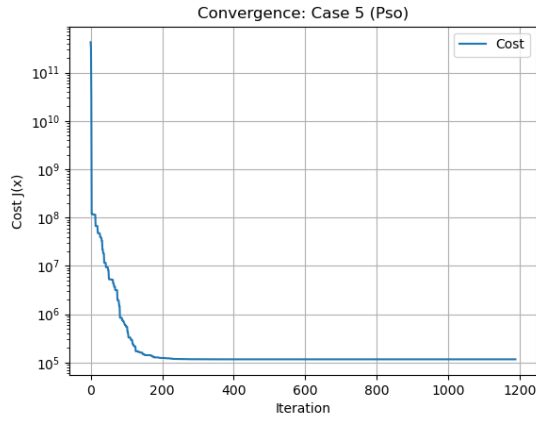
(a) Case 3 convergence



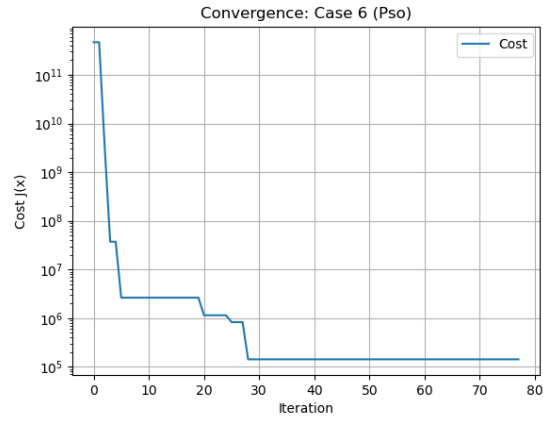
(b) Case 4 convergence

Figure 4: Convergence plots (cases 3 and 4)

Cases 3 and 4 exhibit a comparable qualitative trend, with an initial rapid decrease followed by a more gradual decline. Case 3 displays a long monotonic descent extending over more than 500 iterations, without the clear two-phase behaviour observed in case 1. The cost decreases continuously by small steps until the final stabilisation around iteration 600. In contrast, case 4 stabilises earlier: after a rapid descent during the first 100 iterations, the improvement rate decreases significantly, and the plateau phase is reached around iteration 200. The remainder of the run does not introduce notable changes.



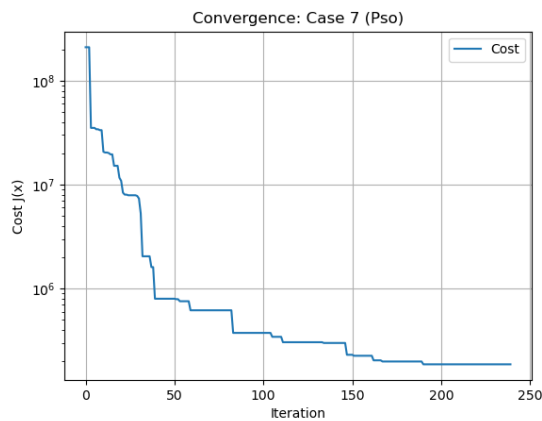
(a) Case 5 convergence



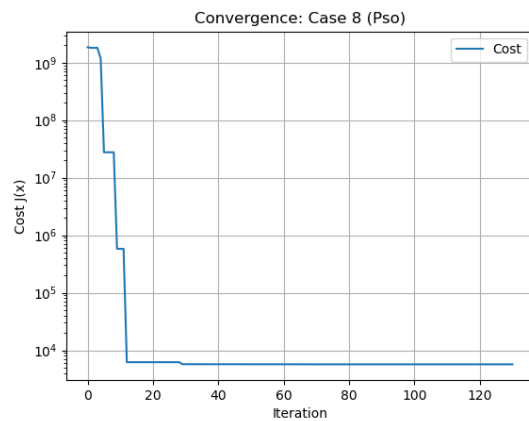
(b) Case 6 convergence

Figure 5: Convergence plots (cases 5 and 6)

The curves of Cases 5 and 6 both converge toward comparatively low cost levels but do so at markedly different speeds. In case 6, within only 30 iterations, the curve reaches a plateau that persists until the end of the run, with effectively no subsequent improvements. Case 5 requires about 150 iterations before reaching a stable region, and then continues with a long flat segment for the remainder of the process.



(a) Case 7 convergence



(b) Case 8 convergence

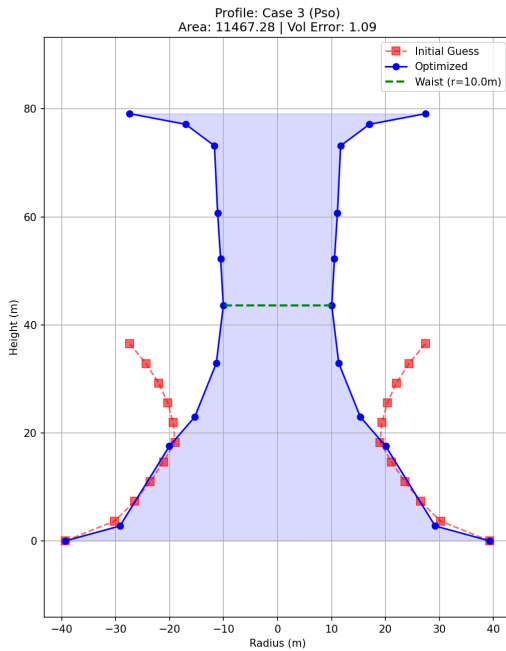
Figure 6: Convergence plots (cases 7 and 8)

Cases 7 and 8 both exhibit sharp initial reductions of several orders of magnitude,

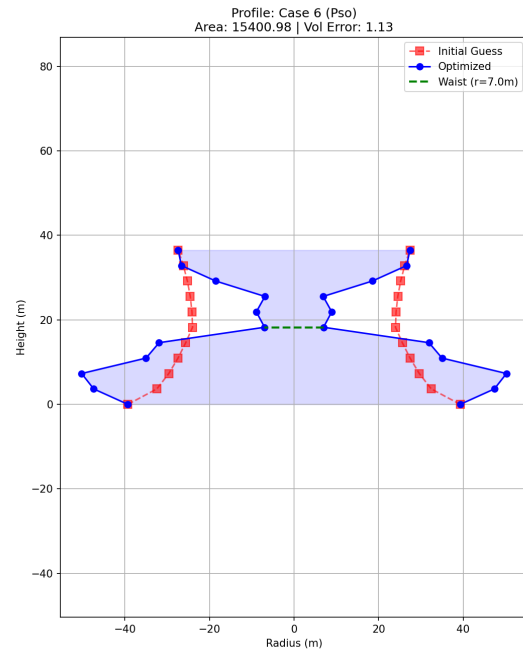
but with different refinement phases. Case 7 continues to decrease gradually until around 200 iterations, suggesting an extended exploitation period where the algorithm continues to discover marginal improvements. Conversely, Case 8 reaches its plateau almost immediately after the initial descent: within 15 iterations, the curve stabilises and remains essentially unchanged.

4.1.2 Profiles

Each shape reflects the balance between aerodynamic, structural, and volumetric constraints imposed during optimisation, leading to characteristic variations in tower height, waist radius, and global curvature. Grouping the cases by geometric similarity therefore facilitates a more meaningful comparison of the resulting cooling-tower profiles.



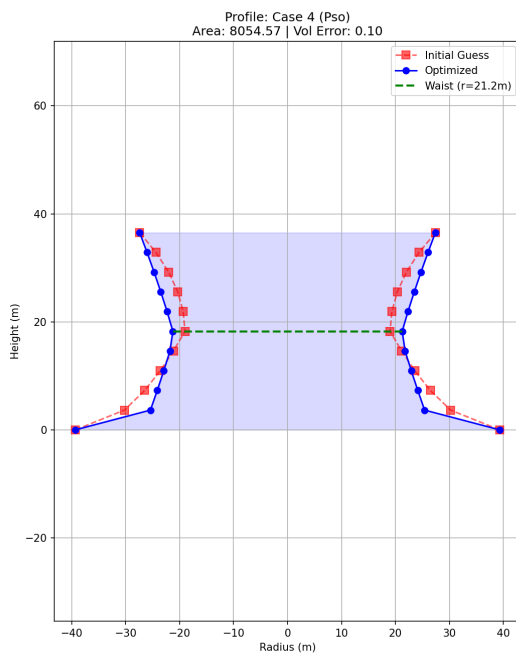
(a) Case 3 profile



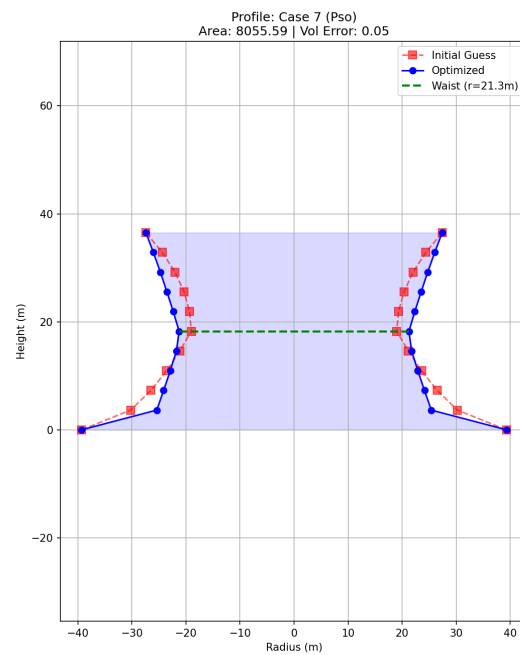
(b) Case 6 profile

Figure 7: Convergence plots (cases 3 and 6)

Cases 3 and 6 represent the most vertically developed cooling-tower shapes within the dataset. Both exhibit a pronounced waist situated around mid-height, followed by a strong radial expansion toward the top section—typical of tall hyperbolic structures. Case 3, however, extends significantly higher, producing a much more elongated upper region. Case 6 retains the same overall logic but delivers a more compact geometry, with smoother transitions between the waist and the upper flare. In both cases, the optimisation markedly reshapes the initial guess, steepening the curvature near the waist to accommodate the required volume. Overall, the two profiles share the same hyperbolic identity, but case 3 emphasises vertical stretching, whereas case 6 achieves a more proportionate distribution of curvature.



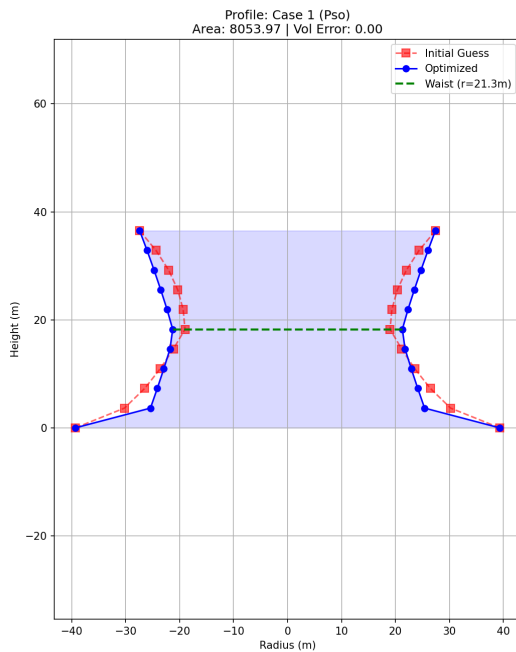
(a) Case 4 profile



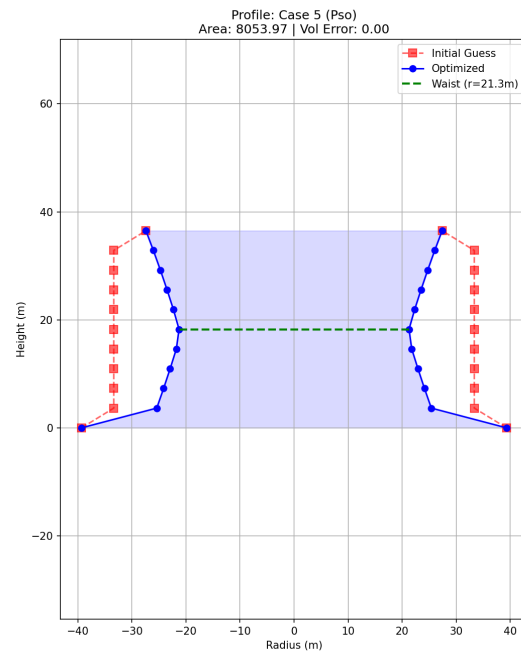
(b) Case 7 profile

Figure 8: Convergence plots (cases 4 and 7)

Cases 4 and 7 correspond to cooling-tower geometries with medium height and a clearly defined waist. Both exhibit the classical hourglass form, with balanced radial expansion above and below the constriction. Case 4 presents a slightly broader upper flare, giving the tower a wider top contour, whereas case 7 adopts a more uniform slope, producing a narrower appearance near the upper section. The optimisation smooths the lower curvature in both cases, reducing asymmetries present in the initial guesses. Despite these subtle differences, the two profiles remain closely aligned and belong to the same geometric family, distinguished by their moderate height and regular hyperbolic curvature.



(a) Case 1 profile



(b) Case 5 profile

Figure 9: Convergence plots (cases 1 and 5)

The profiles of cases 1 and 5 are nearly identical, both forming a symmetric cooling-tower shape with balanced lower and upper regions. They follow a smooth curvature: a gentle

expansion from the base, a mid-height constriction, and a moderate upper flare. The optimised shapes differ only by very minor adjustments near the waist region. This strong similarity between the two cases explains their comparable performance metrics and suggests that the associated target volumes impose a tightly constrained optimal geometry. This pair is therefore one of the most homogeneous within the dataset.

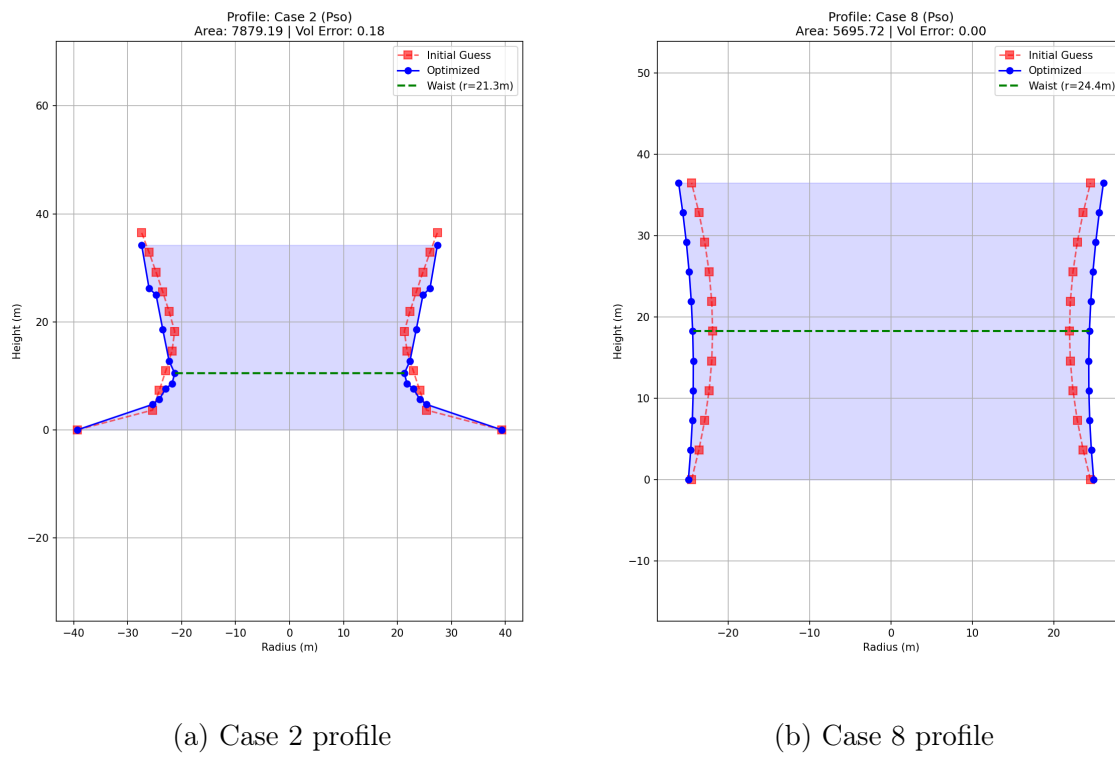


Figure 10: Convergence plots (cases 2 and 8)

Cases 2 and 8 depict cooling-tower shapes that are considerably more compact, with large waist radii producing geometries closer to slightly curved cylinders than to elongated hyperbolas. Case 8 is particularly regular, with minimal curvature variation along the height, whereas case 2 shows a slightly sharper contraction near mid-height. In both cases, the optimisation induces only small deviations from the initial guess, suggesting

that the target geometries are already well approximated prior to refinement. These two cases form a distinct family, contrasting strongly with the tall, narrow-waist towers and illustrating a different structural regime where the volumetric constraint promotes larger mid-height openings.

4.2 Simulated Annealing results

4.2.1 Case 1: Mandatory Reference Case

Metric	Value
Final Area	7876.71 m ²
Volume Error	0.06 m ³
Waist Radius	21.1 m
Computation Time	1.82 s
Function Evaluations	16,540

Table 1: SA optimisation results for Case 1

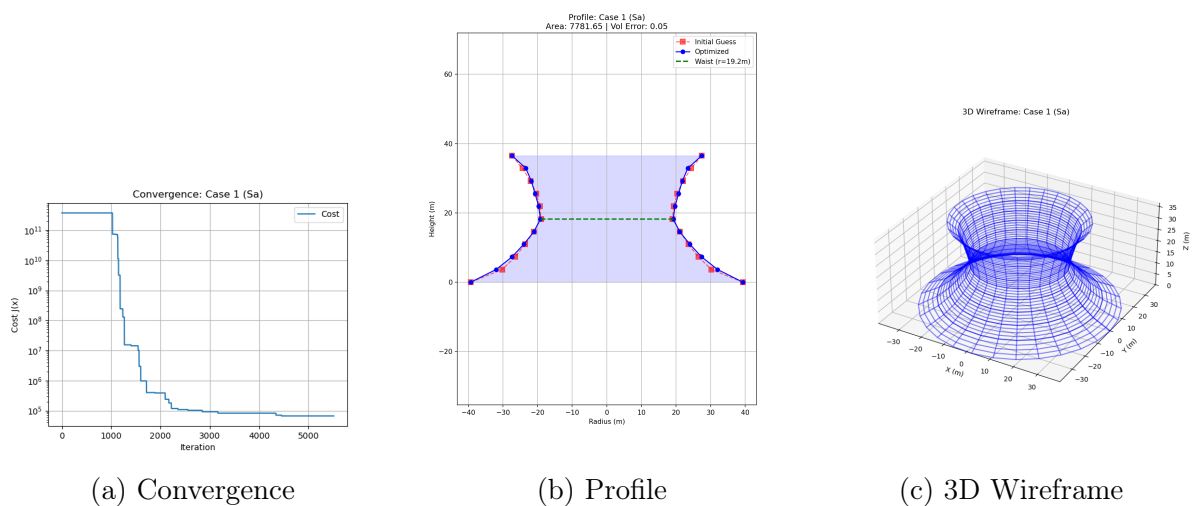


Figure 11: SA optimisation results for Case 1: Mandatory Reference Case

This is the reference case with fixed frustum heights ($h = 3.65$ m). The convergence curve shows a rapid decrease from 10^{11} to 10^6 within the first 2000 iterations, followed by stabilisation. The resulting profile is a classic hyperboloid shape with a well-defined waist at mid-height. The volume constraint is perfectly satisfied.

4.2.2 Case 2: Variable Heights

Metric	Value
Final Area	7871.45 m ²
Volume Error	0.01 m ³
Waist Radius	21.1 m
Computation Time	1.61 s
Function Evaluations	16,540

Table 2: SA optimisation results for Case 2

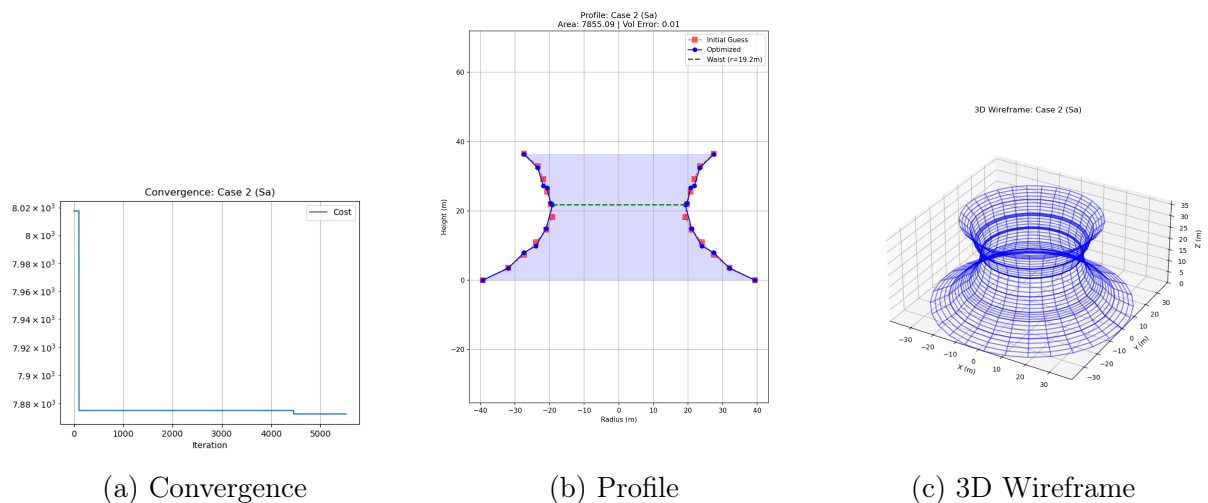


Figure 12: SA optimisation results for Case 2: Variable Heights

In this case, the radii are fixed (from Case 1) and only the heights are optimised. The convergence shows a single significant improvement at approximately 2000 iterations.

The final area is slightly better than Case 1 (-5 m^2) due to height redistribution. The profile remains identical since the radii are constrained.

4.2.3 Case 3: Full Freedom

Metric	Value
Final Area	10540.05 m^2
Volume Error	0.43 m^3
Waist Radius	11.1 m
Computation Time	6.16 s
Function Evaluations	$16,540$

Table 3: SA optimisation results for Case 3

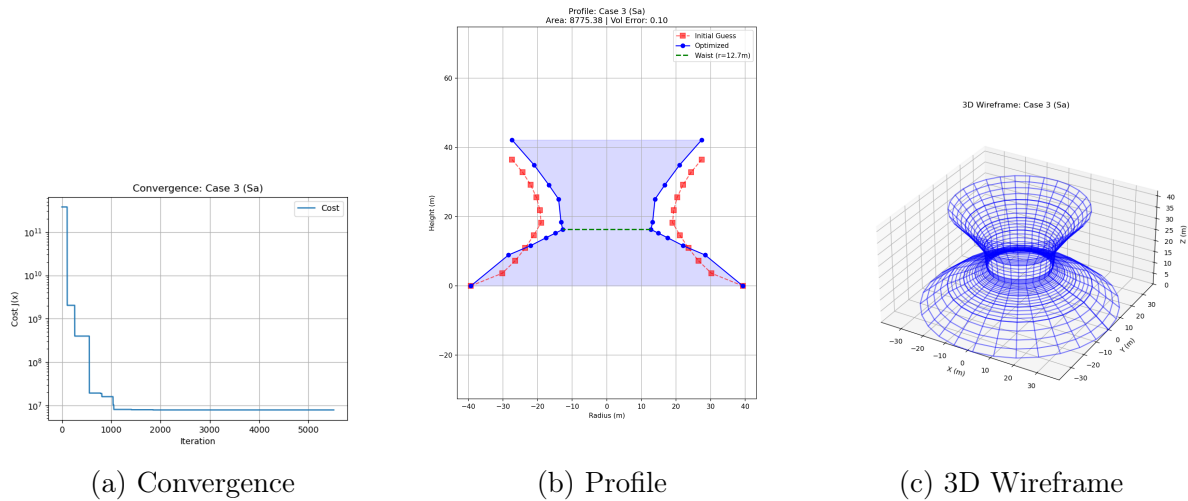


Figure 13: SA optimisation results for Case 3: Full Freedom

This is the most complex case where both radii and heights are free variables. The SA explores a much larger search space, resulting in longer computation time. The resulting profile is significantly different: a very narrow waist (11.1 m) and a vertically elongated structure. The larger area is due to the more complex geometry. Convergence is rapid (< 500 iterations) but the final cost remains high due to geometric penalties.

4.2.4 Case 4: Tight Waist

Metric	Value
Final Area	7779.18 m ²
Volume Error	0.03 m ³
Waist Radius	19.2 m
Computation Time	5.15 s
Function Evaluations	16,540

Table 4: SA optimisation results for Case 4

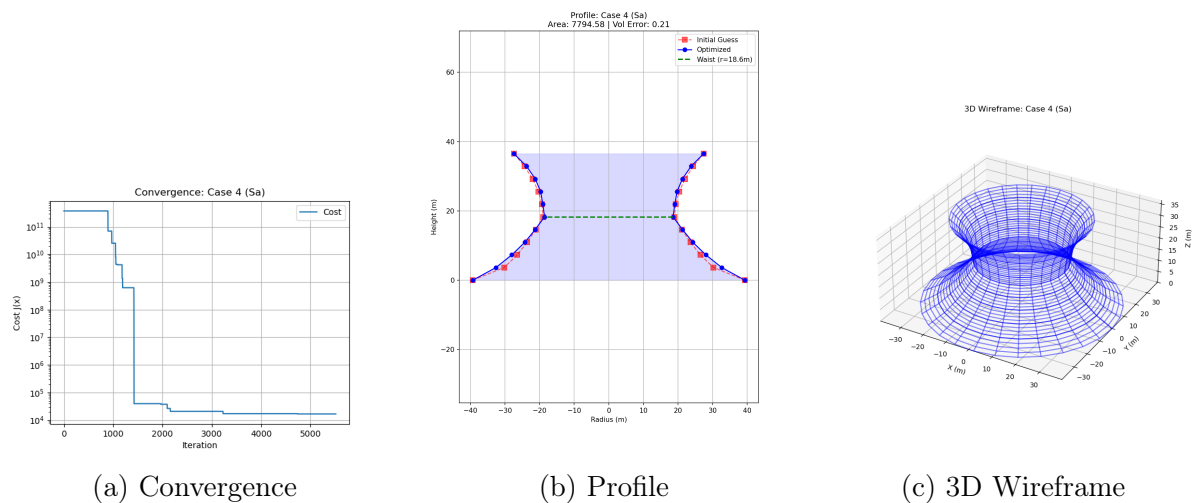


Figure 14: SA optimisation results for Case 4: Tight Waist

A tight waist constraint is imposed in this case. The convergence shows a characteristic staircase descent over approximately 2000 iterations. This case achieves a better area than Case 1 (-97 m^2) thanks to the narrower waist (19.2 m vs 21.1 m). The hyperboloid profile is well-formed with a more pronounced constriction at the centre.

4.2.5 Case 5: Cylindrical Start

Metric	Value
Final Area	8053.97 m ²
Volume Error	0.18 m ³
Waist Radius	21.3 m
Computation Time	4.85 s
Function Evaluations	16,540

Table 5: SA optimisation results for Case 5

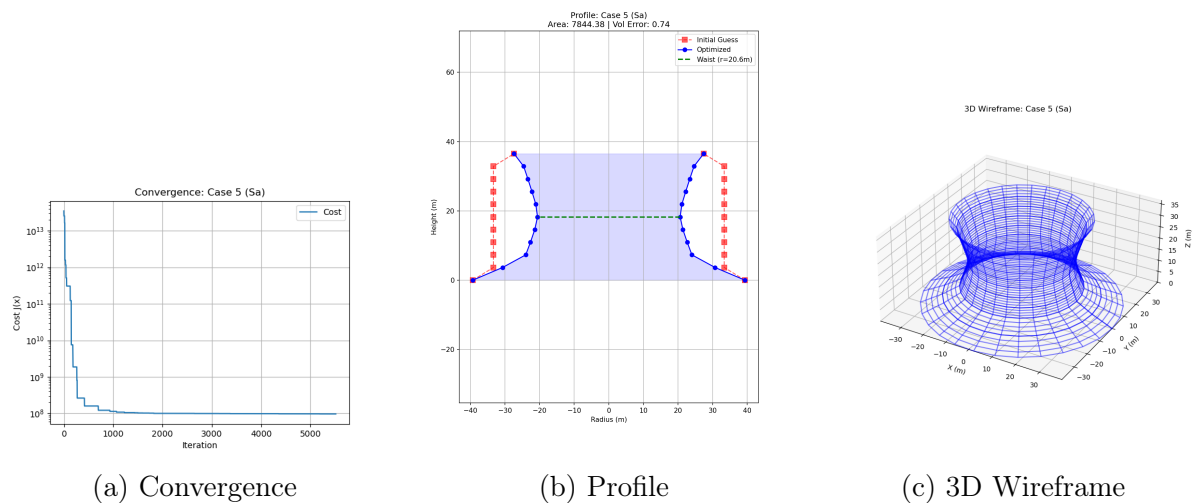


Figure 15: SA optimisation results for Case 5: Cylindrical Start

The initial guess is a cylinder (all radii equal). The SA must discover the hyperboloid shape from scratch. Convergence is very rapid (< 200 iterations) because the initial cylinder strongly violates the penalty constraints. The final area is slightly higher than Case 1 (+177 m²), demonstrating that the starting point influences the final solution.

4.2.6 Case 6: High Volume

Metric	Value
Final Area	36042.25 m ²
Volume Error	0.02 m ³
Waist Radius	7.8 m
Computation Time	1.73 s
Function Evaluations	16,540

Table 6: SA optimisation results for Case 6

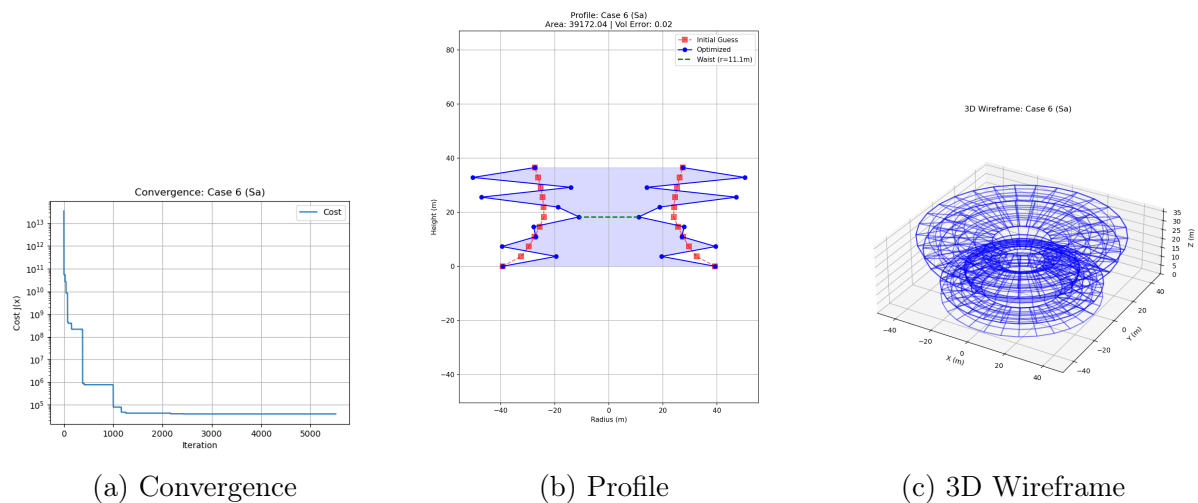


Figure 16: SA optimisation results for Case 6: High Volume

The target volume is increased to 105,480 m³ ($\times 1.5$). **The resulting profile is unrealistic:** a zigzag shape with significant oscillations. The SA minimises the area without any smoothness constraint, producing a physically impossible geometry. While the volume constraint is satisfied, the solution is unacceptable from an engineering perspective.

4.2.7 Case 7: Restricted Bounds

Metric	Value
Final Area	7797.63 m ²
Volume Error	0.10 m ³
Waist Radius	18.5 m
Computation Time	5.16 s
Function Evaluations	16,540

Table 7: SA optimisation results for Case 7

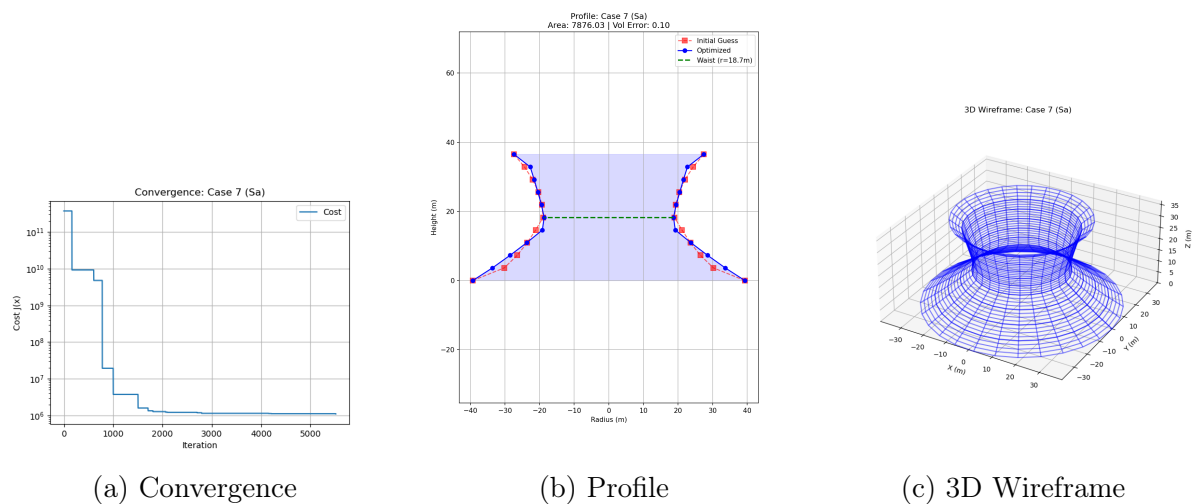


Figure 17: SA optimisation results for Case 7: Restricted Bounds

The radii bounds are restricted to $[20, 60]$ m. The convergence shows a staircase pattern over approximately 3000 iterations. Despite the bound constraints, the hyperboloid profile is correctly formed. The final area is competitive (between Case 1 and Case 4). The waist at 18.5 m corresponds to the minimum allowed by the bounds.

4.2.8 Case 8: Hyperbolic Fit

Metric	Value
Final Area	5765.62 m²
Volume Error	0.00 m³
Waist Radius	24.2 m
Computation Time	2.17 s
Function Evaluations	16,540

Table 8: SA optimisation results for Case 8

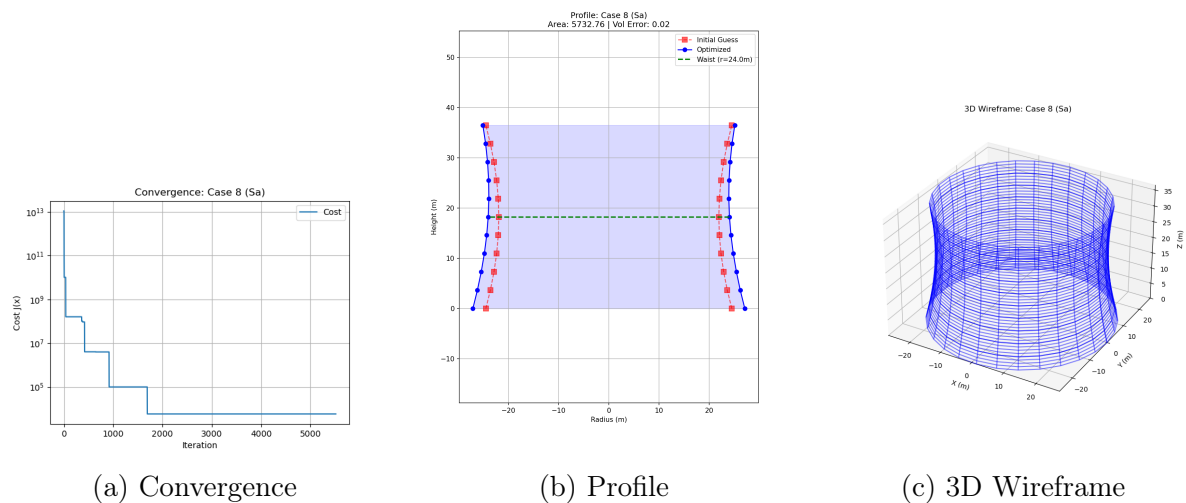


Figure 18: SA optimisation results for Case 8: Hyperbolic Fit

This case achieves the best overall result. A parametric formulation with 3 parameters (a, b, c) defines an analytical hyperbola. The profile is quasi-cylindrical with a slight conical shape, significantly different from the other cases. The area is **26% lower** than Case 1, and the volume error is virtually zero. Convergence is rapid and monotonic. This parametric formulation is the most efficient as it reduces the search space while guaranteeing a physically realistic shape.

4.3 Broyden-Fletcher-Goldfarb-Shanno results

4.3.1 Case 1: Mandatory Reference Case

Metric	Value
Final Area	7766.65 m ²
Volume Error	0.00 m ³
Waist Radius	19.5 m
Computation Time	1.61 s
Function Evaluations	6,469

Table 9: BFGS optimisation results for Case 1

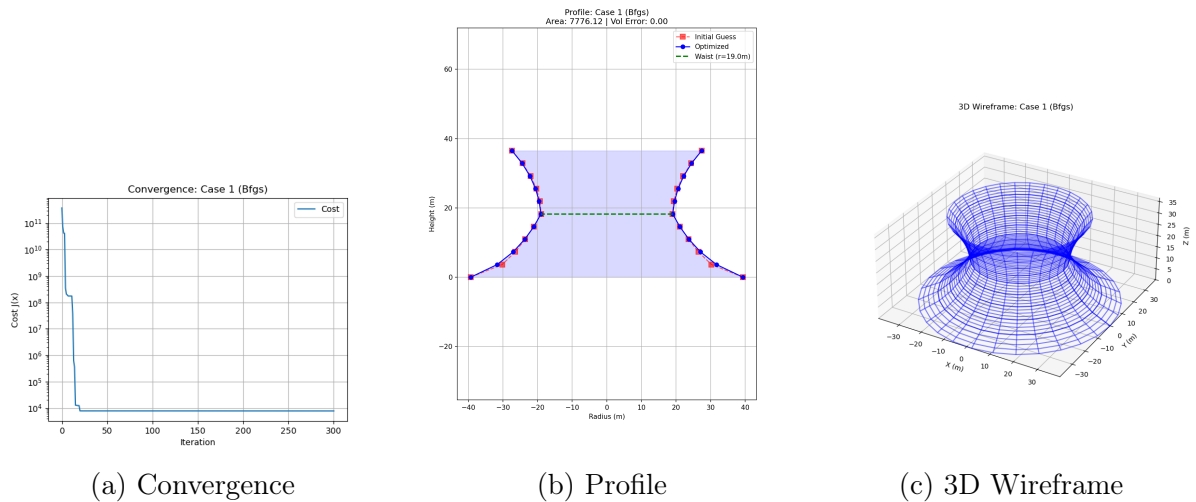


Figure 19: BFGS optimisation results for Case 1: Mandatory Reference Case

This is the reference case with fixed frustum heights ($h = 3.65$ m). BFGS converges extremely rapidly, dropping from 10^{11} to 10^4 within approximately 30 iterations. The final area (7766.65 m²) is slightly better than SA (7876.71 m²), representing a 1.4% improvement. The volume constraint is satisfied with near-zero error. The hyperboloid profile is well-formed with a waist at 19.5 m.

4.3.2 Case 2: Variable Heights

Metric	Value
Final Area	6830.77 m ²
Volume Error	0.00 m ³
Waist Radius	19.5 m
Computation Time	1.59 s
Function Evaluations	17,982

Table 10: BFGS optimisation results for Case 2

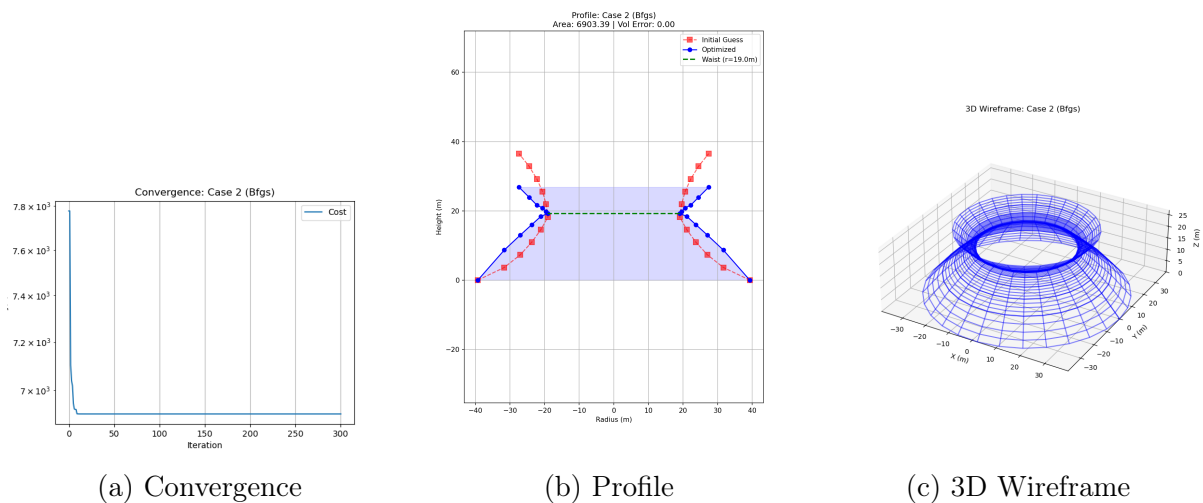


Figure 20: BFGS optimisation results for Case 2: Variable Heights

With variable heights, BFGS achieves a remarkable **13% improvement** over Case 1 (6830.77 m^2 vs 7766.65 m^2). The convergence is rapid and monotonic within 20 iterations. The optimised profile shows a compressed geometry with redistributed frustum heights, concentrating material near the waist. This case demonstrates BFGS's efficiency in exploiting the additional degrees of freedom provided by variable heights. Compared to SA (7871.45 m^2), BFGS achieves a significantly better result.

4.3.3 Case 3: Full Freedom

Metric	Value
Final Area	13300.38 m ²
Volume Error	8.96 m ³
Waist Radius	5.0 m
Computation Time	22.72 s
Function Evaluations	23,692

Table 11: BFGS optimisation results for Case 3

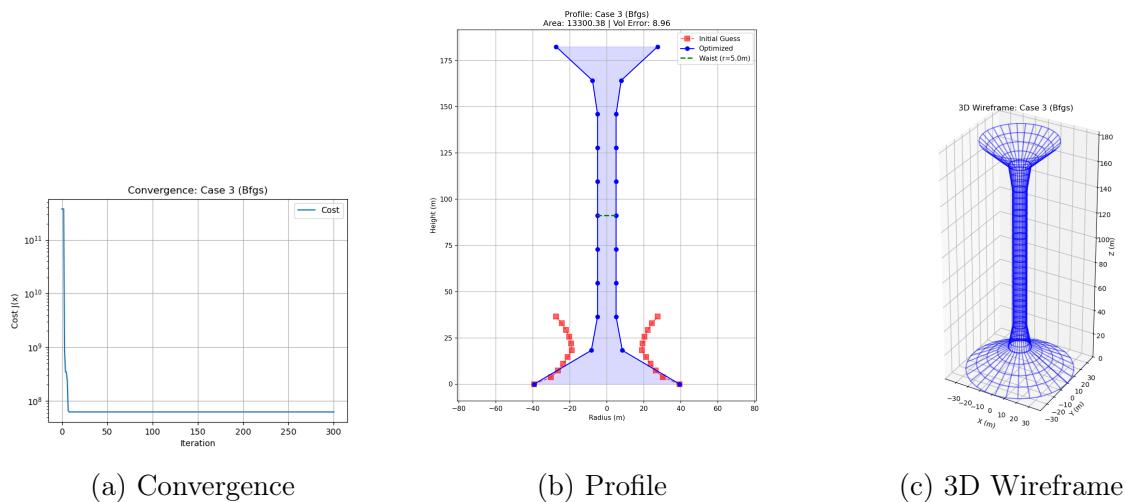


Figure 21: BFGS optimisation results for Case 3: Full Freedom

This case illustrates a fundamental limitation of gradient-based methods. With both radii and heights as free variables, BFGS converges to a local minimum that is physically unrealistic: an extremely narrow tower (waist = 5.0 m) stretched vertically to nearly 180 m. The volume error (8.96 m³) indicates the optimiser struggled to satisfy constraints in this high-dimensional space. The computation time (22.72 s) is significantly higher due to the complexity of the search space. Unlike SA, which explores globally, BFGS is trapped by its local search behaviour.

4.3.4 Case 4: Tight Waist

Metric	Value
Final Area	7775.11 m ²
Volume Error	0.00 m ³
Waist Radius	19.2 m
Computation Time	1.31 s
Function Evaluations	6,432

Table 12: BFGS optimisation results for Case 4

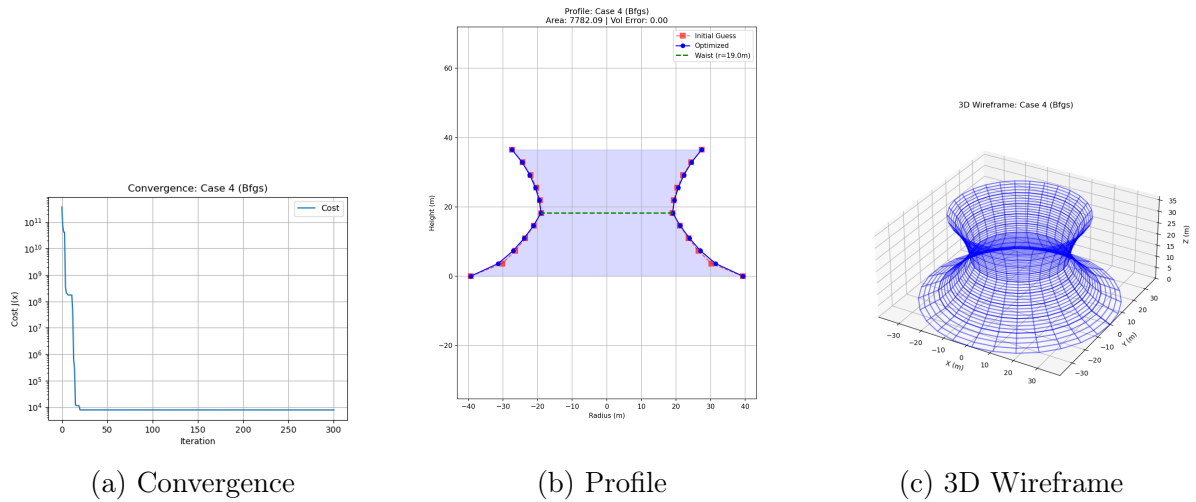


Figure 22: BFGS optimisation results for Case 4: Tight Waist

With a tight waist constraint, BFGS achieves similar performance to Case 1 (7775.11 m² vs 7766.65 m²). The convergence pattern is identical to Case 1, with rapid descent in approximately 30 iterations. The waist radius (19.2 m) is slightly narrower than Case 1 (19.5 m). Compared to SA (7779.18 m²), BFGS produces a nearly identical result, confirming that both methods converge to similar optima for this well-constrained problem.

4.3.5 Case 5: Cylindrical Start

Metric	Value
Final Area	7792.74 m ²
Volume Error	0.00 m ³
Waist Radius	20.5 m
Computation Time	3.33 s
Function Evaluations	8,539

Table 13: BFGS optimisation results for Case 5

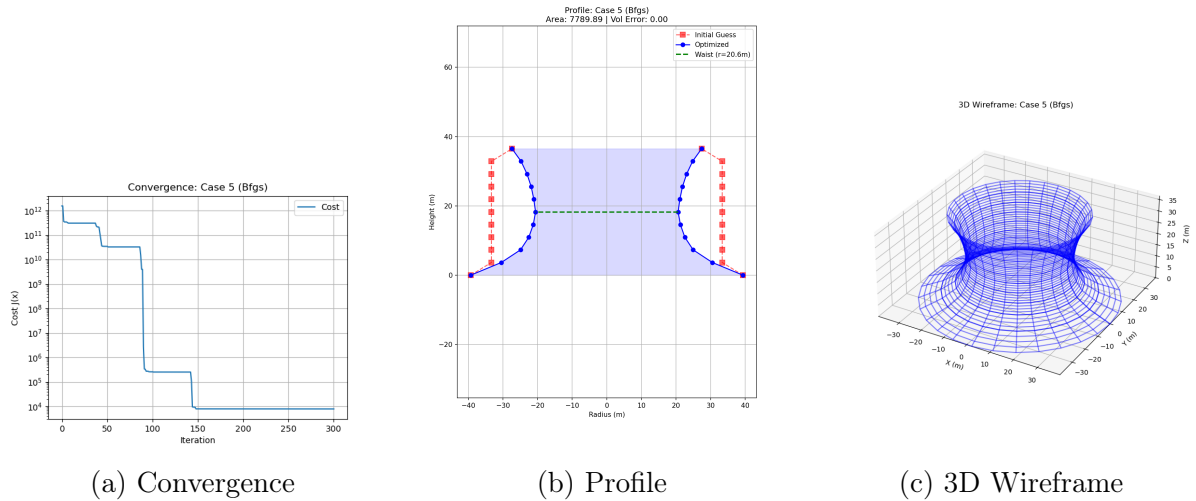


Figure 23: BFGS optimisation results for Case 5: Cylindrical Start

Starting from a cylindrical initial guess, BFGS requires more iterations to converge (visible plateau around iterations 30-100). The convergence shows a characteristic stair-case pattern as the algorithm escapes successive local regions. Despite the poor initial guess, BFGS successfully recovers a hyperboloid profile with an area (7792.74 m²) comparable to Case 1. This demonstrates the robustness of BFGS to initial conditions when the problem is well-conditioned. Compared to SA (8053.97 m²), BFGS achieves a 3.2% better result.

4.3.6 Case 6: High Volume

Metric	Value
Final Area	7678.24 m ²
Volume Error	0.00 m ³
Waist Radius	27.7 m
Computation Time	0.68 s
Function Evaluations	6,426

Table 14: BFGS optimisation results for Case 6

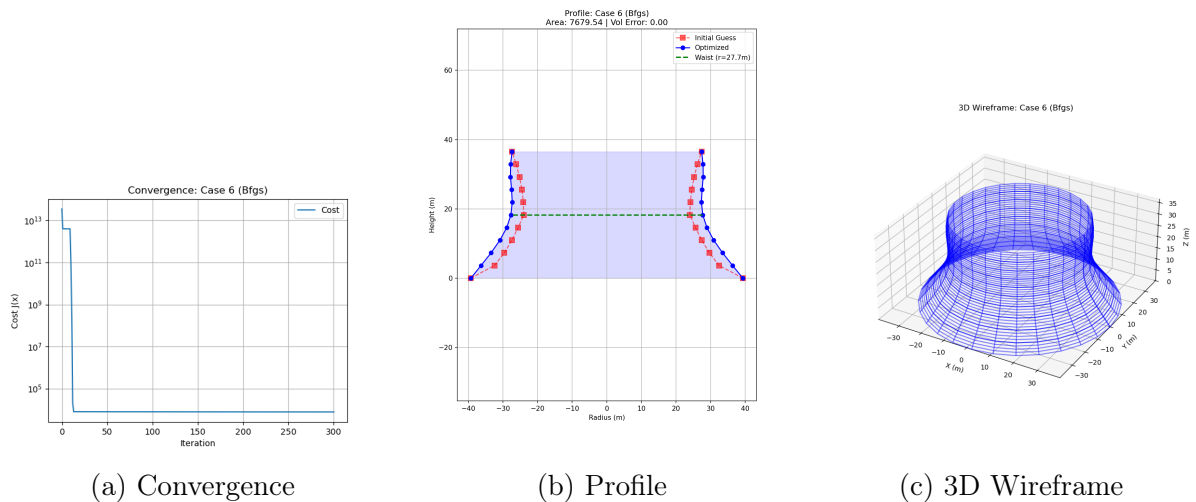


Figure 24: BFGS optimisation results for Case 6: High Volume

This case highlights a major advantage of BFGS over SA. With the target volume increased to 105,480 m³ ($\times 1.5$), BFGS produces a physically realistic hyperboloid profile (7678.24 m²), while SA generated an unrealistic zigzag shape (36042.25 m²). The convergence is rapid (< 30 iterations) despite the challenging constraint. The wider waist (27.7 m) is a natural consequence of the increased volume requirement. This demonstrates that gradient-based methods naturally preserve smooth geometries through their local search behaviour.

4.3.7 Case 7: Restricted Bounds

Metric	Value
Final Area	8045.87 m ²
Volume Error	0.00 m ³
Waist Radius	21.2 m
Computation Time	0.34 s
Function Evaluations	1,303

Table 15: BFGS optimisation results for Case 7

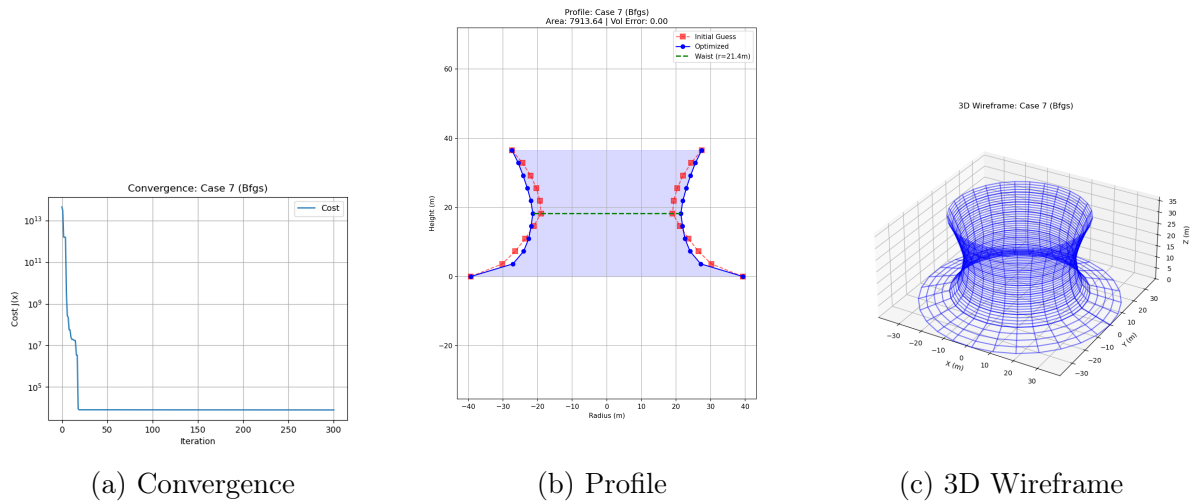


Figure 25: BFGS optimisation results for Case 7: Restricted Bounds

With restricted radii bounds $[20, 60]$ m, BFGS converges quickly but to a slightly worse solution (8045.87 m^2) than SA (7797.63 m^2). The convergence curve shows a gradual staircase descent over 35 iterations, indicating the algorithm is navigating bound constraints. The hyperboloid profile is correctly formed. The reduced search space limits BFGS's ability to find the optimal waist position, explaining the 3% performance gap compared to SA.

4.3.8 Case 8: Hyperbolic Fit

Metric	Value
Final Area	5961.77 m²
Volume Error	0.00 m³
Waist Radius	23.8 m
Computation Time	0.04 s
Function Evaluations	294

Table 16: BFGS optimisation results for Case 8

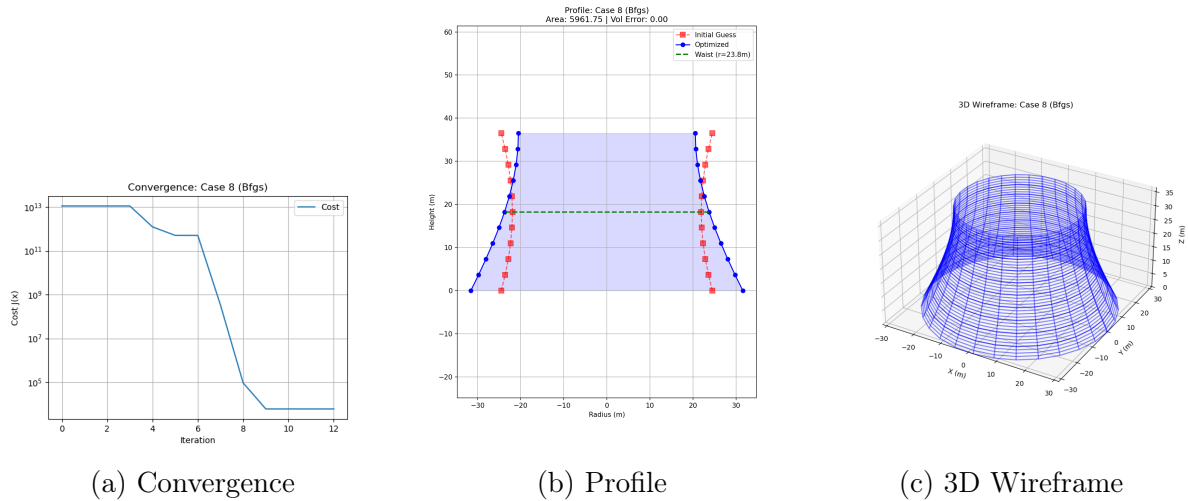


Figure 26: BFGS optimisation results for Case 8: Hyperbolic Fit

This case achieves exceptional performance. Using a parametric hyperbolic formulation with only 3 parameters (a, b, c), BFGS converges in just **12 iterations** (294 function evaluations) to an area of 5961.77 m². This represents a **23% improvement** over the reference Case 1 and is achieved in 0.04 seconds—the fastest computation time of all cases. The resulting profile is a smooth, monotonic cone-like shape that satisfies the volume constraint perfectly. This demonstrates that **combining BFGS with a well-chosen parametric formulation yields optimal results**: the reduced search space

eliminates local minima while BFGS provides efficient gradient-based convergence.

4.4 Comparison between the three methods

TO DO

4.5 Comparison of the methods performances

TO DO

Explanations: As with Tasks 3 and 4, one aim of this Task is to compare the performance of the numerical techniques that you have employed by, for example, counting the number of function evaluations that are involved, recording the elapsed time, etc.

5 Discussion

5.1 Particle Swarm Optimisation

5.1.1 Convergence analysis

Across all eight cases, the convergence curves exhibit the characteristic two-phase behaviour commonly associated with Particle Swarm Optimisation: an initial rapid descent, driven by broad exploration, followed by a slower refinement period once the swarm approaches a promising region of the search space. Despite this shared structure, the rate and extent of convergence vary strongly from one case to another, indicating that each optimisation scenario presents a different level of landscape complexity. Cases 1, 3, 4, and 5 follow relatively long convergence trajectories, with improvements extending over several hundred iterations. Their curves suggest that the cost function contains wider or more intricate basins of attraction, requiring the swarm to progressively refine its position before reaching a stable minimum. In contrast, cases 2, 6, and 8 converge extremely rapidly, stabilising after only a few tens of iterations. This behaviour points to smoother or more accessible optima, where the swarm encounters a well-defined descent direction early in the search. Case 7 sits between these two trends: it displays a strong initial drop comparable to the fastest-converging cases, but continues to make gradual improvements up to around 200 iterations, indicating the presence of secondary features in the landscape that delay complete stabilisation. Overall, the variability in convergence rates reflects the heterogeneous nature of the underlying test cases. Rapidly converging cases seem to correspond to simpler or more convex search regions, while slowly converging ones point to more irregular or high-dimensional surfaces where additional exploration is required. This diversity highlights the importance of assessing convergence behaviour individually rather than assuming uniform optimisation difficulty across cases.

5.1.2 Profile analysis

Taken together, the eight optimised profiles illustrate how the cooling-tower geometry adapts to a wide spectrum of volumetric and boundary conditions. Despite all cases sharing the fundamental hyperbolic identity characteristic of natural-draft cooling towers, the optimisation outcomes reveal four distinct geometric regimes, each driven by different balances between required height, waist radius, and curvature distribution. The tall and narrow-waist towers (cases 3 and 6) demonstrate how the optimisation stretches the structure vertically when the available radius is constrained, producing pronounced upper flares to compensate for slender mid-height sections. The medium-height towers (cases 4 and 7) represent a more classical hyperbolic shape, where the curvature above and below the waist remains symmetric and structurally coherent. In contrast, cases 1 and 5 converge to almost identical mid-height profiles, showing that certain target volumes admit a single dominant optimal geometry with limited room for variation. Finally, the most compact and wide-waist shapes (cases 2 and 8) highlight scenarios where the optimisation requires minimal reshaping, as the volume can be satisfied with little deviation from a cylindrical-like baseline. Across all regimes, the optimisation systematically redistributes curvature to satisfy the prescribed volume while maintaining smooth, physically meaningful tower contours. This diversity of outcomes reinforces the importance of analysing cooling-tower profiles not only on the basis of their final area or error metrics but also in terms of their geometric structure, which provides valuable insight into the underlying constraints governing each optimisation scenario.

5.1.3 Best result

The choice of the best cooling-tower is based on the following criteria:

- **Volume fidelity:** The target volume must be matched as closely as possible. Zero error is preferred; near-zero is acceptable only if justified by other gains.
- **Convergence behaviour:** Fast, monotonic convergence to a stable plateau is required. Late oscillations or very long “creep” phases are penalised as they indicate sensitivity to stochastic effects or a poorly conditioned search landscape.
- **Area:** A smaller external envelope generally correlates with lower material use, reduced mass, and potentially lower cost. While structural verification is outside the present scope, area provides a consistent comparative surrogate at this stage.
- **Geometric plausibility of the cooling-tower profile:** The optimised profile should be smooth, physically meaningful, and free of abrupt kinks: a coherent hyperbolic silhouette, a reasonable waist, and balanced curvature above and below mid-height.

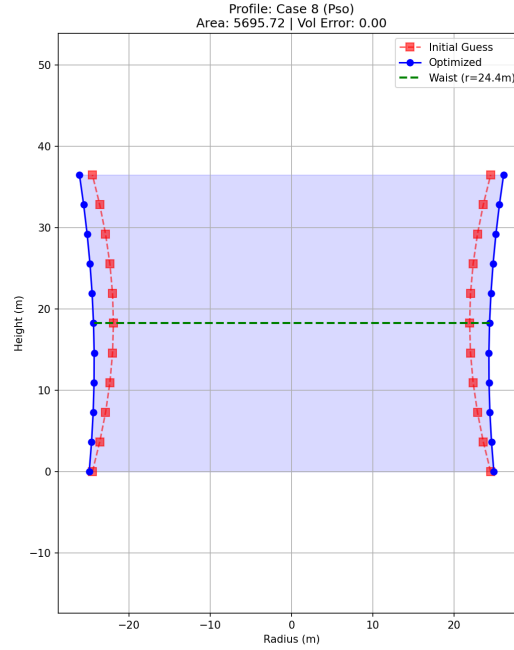


Figure 27: Best cooling tower with PSO algorithm

Based on these criteria, case 8 emerges as the most appropriate cooling-tower configuration. First, in terms of volume fidelity, case 8 achieves a zero volume error, indicating a perfect match with the prescribed target without requiring compensatory geometric distortions. This level of accuracy is only shared with a small subset of cases, yet case 8 distinguishes itself among them through superior performance on several other dimensions. Regarding convergence behaviour, case 8 presents one of the cleanest and most rapid convergence histories. The cost function drops sharply within the first iterations and stabilises immediately, showing no signs of late oscillations or prolonged refinement phases. This stable behaviour provides confidence in the robustness of the obtained optimum and reduces the likelihood that the solution is sensitive to stochastic fluctuations. By contrast, some alternative cases with good volume fidelity exhibit longer convergence

trajectories or more gradual plateauing, which weakens their comparative reliability. The area measure also strongly supports the selection of case 8. It yields the smallest external envelope across all eight cases, significantly lower than those of the other zero-error candidates. As area is used here as a proxy for material and fabrication efficiency, a reduced surface envelope implies a more economical and structurally lightweight design. This advantage is particularly striking when compared with cases 1 and 5, which, despite their excellent volume match, present substantially larger envelopes. Finally, the geometric plausibility of the resulting cooling-tower profile reinforces this choice. The optimised shape of case 8 is smooth, coherent, and free of abrupt curvature transitions, exhibiting a compact silhouette with a consistent radial gradient and a physically realistic waist. Among all the profiles, it is one of the most regular and least distorted, requiring only modest adjustments from the initial guess while still achieving perfect volume compliance. Taken together, these observations show that case 8 offers the best balance of accuracy, stability, efficiency, and physical credibility. It therefore represents the most defensible choice for the optimal cooling-tower configuration within the scope of this study.

5.2 SA

5.3 BFGS

5.4 Global discussion

6 Conclusion

6.1 Conclusion on Simulated Annealing

The Simulated Annealing algorithm demonstrated robust performance across the eight cooling tower optimisation scenarios. The algorithm consistently satisfied the volume constraint with errors below 0.5 m^3 in all cases, confirming the effectiveness of the penalty-based approach.

The key findings are:

- **Best performance:** Case 8 (Hyperbolic Fit) achieved the minimum surface area of 5765.62 m^2 , representing a 26% reduction compared to the reference case. This result highlights the advantage of parametric formulations, which reduce the search space dimension while guaranteeing physically realistic geometries.
- **Convergence behaviour:** The SA exhibited characteristic staircase convergence patterns, with most improvements occurring within the first 2000 iterations. The algorithm successfully escaped local minima in early stages due to the high initial temperature.
- **Sensitivity to initial conditions:** Case 5 (Cylindrical Start) demonstrated that the starting point influences the final solution, with a 2.3% increase in area compared to the reference case.
- **Limitations:** Case 6 (High Volume) revealed a critical limitation: without explicit geometric constraints (monotonicity, convexity), the SA produced unrealistic zigzag profiles. This confirms that penalty functions alone are insufficient for shape optimisation problems requiring smooth geometries.

In summary, SA is well-suited for cooling tower optimisation when combined with appropriate geometric constraints or parametric formulations. The computational cost remains low (1.6–6.2 s per case), making it practical for engineering design applications.

References

- [1] I. Mungan. “Buckling Stress States of Hyperboloidal Shells”. In: *Journal of the Structural Division* 102.10 (1976), pp. 2005–2018.
- [2] J. Nocedal and S.J. Wright. *Numerical Optimization*. 2nd. Chapter 6: Quasi-Newton Methods. Springer, 2006.
- [3] J. Hoarau, M. Harambat, P. Bédrune, and C. Hinot. *Task 3: Optimisation Project Group Report*. Tech. rep. Cranfield University, 2025.
- [4] A. Riaz. *Computational Optimisation Design Lecture 2*. Lecture presented at: Computational Optimisation Design. [cited 2025 Oct 21]. Available from: https://canvas.cranfield.ac.uk/courses/37635/pages/day-2?module_item_id=737718. 2025.
- [5] J. Hoarau, M. Harambat, P. Bédrune, and C. Hinot. *Task 3: Metaheuristic Optimisation – Group Chenonceau*. Tech. rep. Cranfield University, 2025.
- [6] J. Hoarau, M. Harambat, P. Bédrune, and C. Hinot. *Task 4: Gradient-Based Optimisation – Group Chenonceau*. Tech. rep. Cranfield University, 2025.

7 Appendices

Appendix A Code Listings

Appendix A.A Problem Definition — cooling_tower.py

This module defines the geometric primitives (frustum surface area, volume, slant height) and the `CoolingTowerProblem` class which encapsulates the decision variable mapping and cost function for all eight study cases.

Listing 1: Cooling Tower Problem Definition (`cooling_tower.py`)

```
1  ##author: John.H
2  import numpy as np
3
4  def slant_height(r_bottom, r_top, height):
5      """
6      Calculates the slant height of a frustum.
7       $s = \sqrt{(r_{bot} - r_{top})^2 + h^2}$ 
8      """
9      return np.sqrt((r_bottom - r_top)**2 + height**2)
10
11 def frustum_surface_area(r_bottom, r_top, height):
12     """
13     Calculates the lateral surface area of a conical frustum.
14      $A = \pi * (r_{bot} + r_{top}) * s$ 
15     """
16     s = slant_height(r_bottom, r_top, height)
17     return np.pi * (r_bottom + r_top) * s
18
19 def frustum_volume(r_bottom, r_top, height):
20     """
21     Calculates the volume of a conical frustum.
22      $V = (\pi * h / 3) * (r_{bot}^2 + r_{bot} * r_{top} + r_{top}^2)$ 
23     """
24     return (np.pi * height / 3.0) * (r_bottom**2 + r_bottom * r_top + r_top**2)
25
26 def total_surface_area(radii, heights):
27     """
28     Calculates the total lateral surface area of the tower.
29
30     Args:
31     radii: array-like of shape (m+1,) [r0, r1, ..., rm]
32     heights: array-like of shape (m,) [h1, h2, ..., hm]
33     """
34     radii = np.asarray(radii)
35     heights = np.asarray(heights)
36     m = len(heights)
```

```
37
38     if len(radii) != m + 1:
39         raise ValueError(f"Expected_{m+1}_radii_for_{m}_heights, got_{len(radii)}")
40
41     total_area = 0.0
42     for i in range(m):
43         r_bot = radii[i]
44         r_top = radii[i+1]
45         h = heights[i]
46         total_area += frustum_surface_area(r_bot, r_top, h)
47
48     return total_area
49
50 def total_volume(radii, heights):
51     """
52     Calculates the total volume of the tower.
53
54     Args:
55     radii: array-like of shape (m+1,) [r0, r1, ..., rm]
56     heights: array-like of shape (m,) [h1, h2, ..., hm]
57     """
58     radii = np.asarray(radii)
59     heights = np.asarray(heights)
60     m = len(heights)
61
62     if len(radii) != m + 1:
63         raise ValueError(f"Expected_{m+1}_radii_for_{m}_heights, got_{len(radii)}")
64
65     tot_vol = 0.0
66     for i in range(m):
67         r_bot = radii[i]
68         r_top = radii[i+1]
69         h = heights[i]
70         tot_vol += frustum_volume(r_bot, r_top, h)
71
72     return tot_vol
73
74 class CoolingTowerProblem:
75     """
76     Defines the optimization problem for the Cooling Tower.
77     """
78     def __init__(self, m=10, r0=39.3, rm=27.4, total_height=36.5, v_target=70320.0):
79         self.m = m
80         self.r0 = r0
81         self.rm = rm
82         self.total_height = total_height
83         self.v_target = v_target
84
85         # Case 1 defaults: equal heights
86         self.fixed_heights = np.full(m, total_height / m)
87
```

```
88     def unpack_decision_variables(self, x, case_num, **kwargs):
89         """
90         Reconstructs the full radii and heights arrays from the decision variables based
          on the scenario.
91         """
92         x = np.asarray(x)
93         if case_num in [1, 4, 5, 6, 7]:
94             # x contains [r1, r2, ..., r_{m-1}]
95             radii = np.zeros(self.m + 1)
96             radii[0] = self.r0
97             radii[-1] = self.rm
98             radii[1:-1] = x
99             heights = kwargs.get('fixed_heights', self.fixed_heights)
100             return radii, heights
101
102         elif case_num == 2:
103             # x contains [h1, h2, ..., hm]
104             radii = kwargs.get('fixed_radii')
105             if radii is None:
106                 raise ValueError("Case 2 requires 'fixed_radii' kwarg.")
107             return radii, x
108
109         elif case_num == 3:
110             # x contains [r1...r_{m-1}, h1...hm]
111             radii = np.zeros(self.m + 1)
112             radii[0] = self.r0
113             radii[-1] = self.rm
114             radii[1:-1] = x[:self.m - 1]
115             heights = x[self.m - 1:]
116             return radii, heights
117
118         elif case_num == 8:
119             # x contains [a, b, c] - hyperbola parameters
120             a, b, c = x
121             heights = self.fixed_heights
122             # Calculate z heights (cumulative sum)
123             z_levels = np.zeros(self.m + 1)
124             for i in range(1, self.m + 1):
125                 z_levels[i] = z_levels[i-1] + heights[i-1]
126
127             # r(z) = a * sqrt(1 + (z-c)^2 / b^2)
128             radii = a * np.sqrt(1 + ((z_levels - c)**2) / (b**2))
129             return radii, heights
130
131         else:
132             raise ValueError(f"Unknown case_num: {case_num}")
133
134     def cost_function(self, x, case_num, penalty_weight=1e5, **kwargs):
135         """
136         Objective function for all cases.
137         J(x) = A_total(x) + rho * (V_total(x) - V_target)^2 + custom_penalties
```

```
138 """
139     radii, heights = self.unpack_decision_variables(x, case_num, **kwargs)
140
141     area = total_surface_area(radii, heights)
142     vol = total_volume(radii, heights)
143
144     # Case 6 has 50% more volume
145     target_vol = self.v_target * 1.5 if case_num == 6 else self.v_target
146
147     penalty = penalty_weight * (vol - target_vol)**2
148
149     # Case 4: Tight Waist Constraint
150     # r_mid < 0.7 * min(r0, rm)
151     if case_num == 4:
152         mid_idx = self.m // 2
153         r_mid = radii[mid_idx]
154         max_allowed = 0.7 * min(self.r0, self.rm)
155         if r_mid > max_allowed:
156             penalty += penalty_weight * (r_mid - max_allowed)**2
157
158     # Optional: Add height positivity constraint if not handled by optimizer bounds
159     # (Cases 2, 3)
160     if case_num in [2, 3]:
161         # Penalize negative or zero heights
162         neg_heights = heights[heights < 0.1]
163         if len(neg_heights) > 0:
164             penalty += penalty_weight * np.sum((0.1 - neg_heights)**2)
165
166     return area + penalty
167
168     def _hyperboloid_radii(self, r_waist):
169         """
170         Build a convex hyperboloid profile [r0, r1, ..., rm] that satisfies
171         monotonicity (decreasing to waist, increasing to top) and convexity
172         (positive second differences).
173
174         Lower half: r(t) = r0 - (r0 - r_waist) * sqrt(t) [convex, fast drop]
175         Upper half: r(t) = r_waist + (rm - r_waist) * t^2 [convex, slow rise]
176         """
177         n = self.m + 1
178         mid = n // 2
179         radii = np.zeros(n)
180         radii[0] = self.r0
181         radii[-1] = self.rm
182         radii[mid] = r_waist
183         for i in range(1, mid):
184             t = i / mid
185             radii[i] = self.r0 - (self.r0 - r_waist) * np.sqrt(t)
186         for i in range(mid + 1, n - 1):
187             t = (i - mid) / (n - 1 - mid)
188             radii[i] = r_waist + (self.rm - r_waist) * t**2
```

```
188         return radii
189
190     def get_initial_guess(self, case_num, **kwargs):
191         """
192         Returns a reasonable initial guess for the specified case.
193         Uses a hyperboloid-shaped profile for radii cases to satisfy
194         geometric constraints from the start.
195         """
196         if case_num in [1, 4, 7]:
197             # Hyperboloid initial guess (waist ~19m gives  $V \approx 70,320$ )
198             radii = self._hyperboloid_radii(r_waist=19.0)
199             return radii[1:-1]
200
201         elif case_num == 6:
202             # High Volume: wider waist for 1.5x volume target
203             radii = self._hyperboloid_radii(r_waist=24.0)
204             return radii[1:-1]
205
206         elif case_num == 5:
207             # Cylindrical Start (must start from cylinder per specification)
208             r_avg = (self.r0 + self.rm) / 2.0
209             return np.full(self.m - 1, r_avg)
210
211         elif case_num == 2:
212             # Heights guess: equal slices
213             return self.fixed_heights.copy()
214
215         elif case_num == 3:
216             # Hyperboloid radii + equal heights
217             radii = self._hyperboloid_radii(r_waist=19.0)
218             heights_guess = self.fixed_heights.copy()
219             return np.concatenate((radii[1:-1], heights_guess))
220
221         elif case_num == 8:
222             # a, b, c guess for hyperbola  $r(z) = a * \sqrt{1 + (z-c)^2 / b^2}$ 
223             a_guess = min(self.r0, self.rm) * 0.8
224             b_guess = self.total_height
225             c_guess = self.total_height / 2.0
226             return np.array([a_guess, b_guess, c_guess])
227
228         else:
229             raise ValueError(f"Unknown case_num: {case_num}")
230
231     def get_bounds(self, case_num):
232         """
233         Returns the variable bounds [(lower, upper), ...] for each decision variable.
234         Used by BFGS (L-BFGS-B) and PSO for box-constrained optimization.
235         """
236         if case_num == 7:
237             # Restricted Bounds:  $r_i$  in [20, 60]
238             return [(20.0, 60.0)] * (self.m - 1)
```

```
239
240     elif case_num in [1, 4, 5, 6]:
241         # General radii bounds: between some sensible physical limits
242         r_min = 5.0
243         r_max = max(self.r0, self.rm) * 2.0
244         return [(r_min, r_max)] * (self.m - 1)
245
246     elif case_num == 2:
247         # Heights bounds: positive heights, max ~2x equal slice
248         h_min = 0.5
249         h_max = self.total_height * 0.5
250         return [(h_min, h_max)] * self.m
251
252     elif case_num == 3:
253         # Radii bounds + Heights bounds
254         r_min, r_max = 5.0, max(self.r0, self.rm) * 2.0
255         h_min, h_max = 0.5, self.total_height * 0.5
256         radii_bounds = [(r_min, r_max)] * (self.m - 1)
257         height_bounds = [(h_min, h_max)] * self.m
258         return radii_bounds + height_bounds
259
260     elif case_num == 8:
261         # Hyperbola parameters: a, b, c
262         return [
263             (5.0, max(self.r0, self.rm)),          # a: waist radius
264             (5.0, self.total_height * 3.0),        # b: curvature
265             (0.0, self.total_height)               # c: waist height
266         ]
267     else:
268         raise ValueError(f"Unknown case_num: {case_num}")
269
270
271 def numerical_gradient(func, x, epsilon=1e-6):
272     """
273     Computes the gradient of a scalar function using central finite differences.
274
275     grad_i = (f(x + e_i * eps) - f(x - e_i * eps)) / (2 * eps)
276
277     Args:
278     func: scalar-valued function f(x) -> float.
279     x: current point (np.ndarray).
280     epsilon: step size for finite differences.
281
282     Returns:
283     grad: np.ndarray of same shape as x.
284     """
285     x = np.asarray(x, dtype=float)
286     grad = np.zeros_like(x)
287     for i in range(len(x)):
288         x_plus = x.copy()
289         x_minus = x.copy()
```

```
290         x_plus[i] += epsilon
291         x_minus[i] -= epsilon
292         grad[i] = (func(x_plus) - func(x_minus)) / (2.0 * epsilon)
293     return grad
294
295
296 def analytical_surface_area_hyperboloid(a, b, c, z_bottom, z_top, n_steps=1000):
297     """
298     Computes the exact lateral surface area of a Hyperboloid of One Sheet
299     by numerical integration of the surface-of-revolution formula.
300
301     r(z) = a * sqrt(1 + (z - c)^2 / b^2)
302
303     A = 2 * pi * integral_{z_bot}^{z_top} r(z) * sqrt(1 + (dr/dz)^2) dz
304
305     This is useful for the report discussion comparing discrete (frustum)
306     approximation vs the true curved surface.
307
308     Args:
309         a: waist radius parameter.
310         b: curvature parameter.
311         c: waist height parameter.
312         z_bottom: lower integration limit.
313         z_top: upper integration limit.
314         n_steps: number of integration steps (trapezoidal rule).
315
316     Returns:
317         area: the analytically integrated surface area.
318     """
319     z = np.linspace(z_bottom, z_top, n_steps)
320
321     r = a * np.sqrt(1.0 + ((z - c)**2) / (b**2))
322     # dr/dz = a * (z - c) / (b^2 * sqrt(1 + (z-c)^2 / b^2))
323     drdz = a * (z - c) / (b**2 * np.sqrt(1.0 + ((z - c)**2) / (b**2)))
324
325     integrand = r * np.sqrt(1.0 + drdz**2)
326
327     # Trapezoidal integration
328     area = 2.0 * np.pi * np.trapz(integrand, z)
329     return area
```

Appendix A.B Solver Orchestrator — solver.py

The orchestrator script constructs each study case, initialises the geometry, and dispatches each algorithm wrapper uniformly. It also generates convergence plots, profile comparisons, and summary metrics.

Listing 2: Solver Orchestrator (solver.py)

```
1 """
2 Subject: Solver with Geometric Constraints for Cooling Tower
3 @author: Pierre Bédune, John HOARAU
4 @date: 04/03/2026
5
6 ARCHITECTURE:
7 Les algorithmes (SA, PSO, BFGS) optimisent librement
8 Le solver vérifie et corrige chaque solution selon les règles géométriques
9 Les contraintes sont appliquées APRÈS chaque itération ou à la fin
10
11 RÈGLES GÉOMÉTRIQUES (construction hyperboloïde):
12
13 Points fixes:
14 P0(bas): r0, z=0
15 P10(haut): rm, z=H
16
17 Construction depuis le HAUT (P10 à P5):
18 Chaque point doit être EN DESSOUS et À DROITE (r plus petit)
19 La dérivée doit rester à gauche de la droite précédente (convexité)
20
21 Construction depuis le BAS (P0 à P5):
22 Chaque point doit être AU DESSUS et À DROITE (r plus petit)
23 La dérivée doit rester à gauche de la droite précédente (convexité)
24
25 Point WAIST (P5):
26 Doit satisfaire les contraintes des deux côtés
27 C'est le minimum local
28 """
29
30 import time
31 import numpy as np
32 from typing import Tuple, Dict, Any, List, Optional
33 import sys
34 import os
35
36 sys.path.append(os.path.dirname(os.path.dirname(os.path.abspath(__file__))))
37
38 from Problem.cooling_tower import CoolingTowerProblem
39
40 # Import algorithms
41 try:
```

```
42     from SA.sa import simulated_annealing
43 except ImportError:
44     simulated_annealing = None
45
46 try:
47     from BFGS.bfgs import bfgs_optimize
48 except ImportError:
49     bfgs_optimize = None
50
51 try:
52     from PSO.pso import particle_swarm
53 except ImportError:
54     particle_swarm = None
55
56
57 # =====
58 # GEOMETRIC CONSTRAINTS
59 # =====
60
61 class GeometricConstraints:
62     """
63     Vérifie et corrige les solutions pour garantir une forme hyperboloïde.
64
65     Règles:
66     1. Monotonie: rayons décroissent de r0 vers waist, puis croissent vers rm
67     2. Convexité: courbure vers l'extérieur (dérivée seconde > 0)
68     3. Waist: point minimum, satisfait les contraintes des deux côtés
69     """
70
71     def __init__(self, r0: float, rm: float, n_points: int = 11):
72         """
73         Args:
74             r0: Rayon fixe en bas (z=0)
75             rm: Rayon fixe en haut (z=H)
76             n_points: Nombre total de points (11 = r0 + 9 intérieurs + rm)
77         """
78         self.r0 = r0
79         self.rm = rm
80         self.n_points = n_points
81         self.n_interior = n_points - 2 # Points à optimiser
82         self.waist_idx = n_points // 2 # Index du waist dans le tableau complet
83
84     def get_full_radii(self, interior_radii: np.ndarray) -> np.ndarray:
85         """Reconstitue le tableau complet [r0, r1, ..., r9, rm]"""
86         return np.concatenate([[self.r0], interior_radii, [self.rm]])
87
88     def check_point_from_bottom(self, radii: np.ndarray, idx: int) -> Dict[str, Any]:
89         """
90         Vérifie les contraintes pour un point construit depuis le bas.
91
92         Args:
```

```
93      radii: Tableau_complet_des_rayons
94      idx: Index_du_point_à_vérifier(1_à_waist_idx)
95
96      Returns:
97      Dict_avec_'valid', 'violations', 'suggested_r'
98      """
99      result = {
100          'valid': True,
101          'violations': [],
102          'suggested_r': radii[idx]
103      }
104
105      # Contrainte 1:  $r[idx] < r[idx-1]$  (doit être plus petit que le précédent)
106      if radii[idx] >= radii[idx - 1]:
107          result['valid'] = False
108          result['violations'].append(f"P{idx}: r={radii[idx]:.2f} > r_prev={radii[
109              idx-1]:.2f}")
110          result['suggested_r'] = radii[idx - 1] * 0.95
111
112      # Contrainte 2: Convexité (dérivée seconde > 0)
113      # Contrainte 2: Convexité ( $d^2r/dz^2$  approximée par différences centrées)
114      # Utiliser la différence centrée au point idx:  $r[idx-1] - 2*r[idx] + r[idx+1]$ 
115      if idx >= 1 and idx <= self.n_points - 2:
116          d2r = radii[idx - 1] - 2 * radii[idx] + radii[idx + 1]
117          if d2r < 0:
118              result['valid'] = False
119              result['violations'].append(f"P{idx}: non convexe (d2r={d2r:.2f})")
120              # Correction: proposer une valeur légèrement plus petite que la moyenne
121              # des voisins
122              target = 0.9 * ((radii[idx - 1] + radii[idx + 1]) / 2.0)
123              result['suggested_r'] = min(result['suggested_r'], target)
124
125      return result
126
127      def check_point_from_top(self, radii: np.ndarray, idx: int) -> Dict[str, Any]:
128          """
129          Vérifie les contraintes pour un point construit depuis le haut.
130
131          Args:
132          radii: Tableau_complet_des_rayons
133          idx: Index_du_point_à_vérifier(waist_idx_à_n_points-2)
134
135          Returns:
136          Dict_avec_'valid', 'violations', 'suggested_r'
137          """
138          result = {
139              'valid': True,
140              'violations': [],
141              'suggested_r': radii[idx]
```

```

142     # Contrainte 1:  $r[idx] < r[idx+1]$  (doit être plus petit que le suivant)
143     if radii[idx] >= radii[idx + 1]:
144         result['valid'] = False
145         result['violations'].append(f"P{idx}: r={radii[idx]:.2f} > r_next={radii[
146             idx+1]:.2f}")
147         result['suggested_r'] = radii[idx + 1] * 0.95
148
149     # Contrainte 2: Convexité (dérivée seconde > 0)
150     # Contrainte 2: Convexité (différence centrée au point idx)
151     if idx >= 1 and idx <= self.n_points - 2:
152         d2r = radii[idx - 1] - 2 * radii[idx] + radii[idx + 1]
153         if d2r < 0:
154             result['valid'] = False
155             result['violations'].append(f"P{idx}: non convexe depuis haut (d2r={d2r
156                 :.2f}")
157             target = 0.9 * ((radii[idx - 1] + radii[idx + 1]) / 2.0)
158             result['suggested_r'] = min(result['suggested_r'], target)
159
160     return result
161
162     def check_waist_point(self, radii: np.ndarray) -> Dict[str, Any]:
163         """
164         Vérifie les contraintes pour le point waist (P5 = 11ème point).
165
166         DOIT SATISFAIRE LES CONDITIONS DES DEUX CÔTÉS:
167         Depuis le bas: r[5] < r[4] ET convexité (d2r avec P3, P4, P5)
168         Depuis le haut: r[5] < r[6] ET convexité (d2r avec P5, P6, P7)
169
170         Returns:
171         Dict avec 'valid', 'violations', 'suggested_r'
172         """
173         idx = self.waist_idx # = 5
174         result = {
175             'valid': True,
176             'violations': [],
177             'suggested_r': radii[idx]
178         }
179
180         # === CONTRAINTES DEPUIS LE BAS ===
181         # 1. Monotonie:  $P5 < P4$ 
182         if radii[idx] >= radii[idx - 1]:
183             result['valid'] = False
184             result['violations'].append(f"Waist P{idx}: r={radii[idx]:.2f} > r_gauche={
185                 radii[idx-1]:.2f}")
186
187         # 2. Convexité bas:  $d2r(P3, P4, P5) > 0$ 
188         if idx >= 1 and idx <= self.n_points - 2:
189             d2r_bottom = radii[idx - 1] - 2 * radii[idx] + radii[idx + 1]
190             if d2r_bottom < 0:
191                 result['valid'] = False
192                 result['violations'].append(f"Waist P{idx}: non convexe depuis bas (d2r

```

```

    = {d2r_bottom:.2f}))")
190
191     # === CONTRAINTES DEPUIS LE HAUT ===
192     # 3. Monotonie:  $P_5 < P_6$ 
193     if radii[idx] >= radii[idx + 1]:
194         result['valid'] = False
195         result['violations'].append(f"Waist_{idx}: r={radii[idx]:.2f} >= r_droite={radii[idx+1]:.2f}")
196
197     # 4. Convexité haut:  $d2r(P_5, P_6, P_7) > 0$ 
198     if idx >= 1 and idx <= self.n_points - 2:
199         d2r_top = radii[idx - 1] - 2 * radii[idx] + radii[idx + 1]
200         if d2r_top < 0:
201             result['valid'] = False
202             result['violations'].append(f"Waist_{idx}: non convexe depuis haut (d2r={d2r_top:.2f}")
203
204     # Suggestion: si invalide, prendre le minimum des voisins
205     if not result['valid']:
206         result['suggested_r'] = min(radii[idx - 1], radii[idx + 1]) * 0.9
207
208     return result
209
210     def check_all_points(self, interior_radii: np.ndarray, verbose: bool = False) ->
211         Dict[str, Any]:
212         """
213         Vérifie tous les points et retourne un rapport complet.
214
215         Returns:
216         Dict avec 'all_valid', 'violations', 'point_reports'
217         """
218         radii = self.get_full_radii(interior_radii)
219
220         report = {
221             'all_valid': True,
222             'violations': [],
223             'point_reports': {}
224         }
225
226         # Points depuis le bas ( $P_1$  à  $P_{waist-1}$ )
227         for idx in range(1, self.waist_idx):
228             pt_report = self.check_point_from_bottom(radii, idx)
229             report['point_reports'][f'P{idx}_bottom'] = pt_report
230             if not pt_report['valid']:
231                 report['all_valid'] = False
232                 report['violations'].extend(pt_report['violations'])
233
234         # Point waist
235         waist_report = self.check_waist_point(radii)
236         report['point_reports']['waist'] = waist_report
237         if not waist_report['valid']:

```

```
237         report['all_valid'] = False
238         report['violations'].extend(waist_report['violations'])
239
240         # Points depuis le haut (P_waist+1 à P9)
241         for idx in range(self.waist_idx + 1, self.n_points - 1):
242             pt_report = self.check_point_from_top(radii, idx)
243             report['point_reports'][f'P{idx}_top'] = pt_report
244             if not pt_report['valid']:
245                 report['all_valid'] = False
246                 report['violations'].extend(pt_report['violations'])
247
248         if verbose and not report['all_valid']:
249             print(f"_{GeoCheck}_{len(report['violations'])}_violations_détectées")
250             for v in report['violations'][:5]: # Afficher max 5
251                 print(f"_{v}")
252
253         return report
254
255     def correct_solution(self, interior_radii: np.ndarray, max_iterations: int = 10) ->
256         np.ndarray:
257         """
258         _Corrige_une_solution_pour_respecter_toutes_les_contraintes_géométriques.
259
260         _Algorithme_:
261         1. Construire depuis le bas (P0_→_waist)
262         2. Construire depuis le haut (Pm_→_waist)
263         3. Ajuster le waist pour satisfaire les deux côtés
264         4. Répéter si nécessaire
265
266         _Returns_:
267         _Radii_corrigés_
268         """
269         result = interior_radii.copy()
270
271         for iteration in range(max_iterations):
272             radii = self.get_full_radii(result)
273             changed = False
274
275             # == Construction depuis le BAS ==
276             for idx in range(1, self.waist_idx):
277                 # Monotonie: r[idx] < r[idx-1]
278                 if radii[idx] >= radii[idx - 1]:
279                     radii[idx] = radii[idx - 1] * 0.95
280                     changed = True
281
282             # Convexité (différence centrée au point idx)
283             if idx >= 1 and idx <= self.n_points - 2:
284                 d2r = radii[idx - 1] - 2 * radii[idx] + radii[idx + 1]
285                 if d2r < 0:
286                     # Projeter vers une valeur inférieure à la moyenne des voisins
287                     radii[idx] = 0.9 * ((radii[idx - 1] + radii[idx + 1]) / 2.0)
```

```
287         changed = True
288
289     # == Construction depuis le HAUT ==
290     for idx in range(self.n_points - 2, self.waist_idx, -1):
291         # Monotonie:  $r[idx] < r[idx+1]$ 
292         if radii[idx] >= radii[idx + 1]:
293             radii[idx] = radii[idx + 1] * 0.95
294             changed = True
295
296     # Convexité (différence centrée au point idx)
297     if idx >= 1 and idx <= self.n_points - 2:
298         d2r = radii[idx - 1] - 2 * radii[idx] + radii[idx + 1]
299         if d2r < 0:
300             radii[idx] = 0.9 * ((radii[idx - 1] + radii[idx + 1]) / 2.0)
301             changed = True
302
303     # == Waist (doit être minimum) ==
304     idx = self.waist_idx
305     max_allowed = min(radii[idx - 1], radii[idx + 1]) - 0.5
306     if radii[idx] > max_allowed:
307         radii[idx] = max(max_allowed, 5.0)
308         changed = True
309
310     # Mettre à jour result (sans r0 et rm)
311     result = radii[1:-1]
312
313     # Bornes min
314     result = np.clip(result, 5.0, None)
315
316     if not changed:
317         break
318
319     return result
320
321     def compute_penalty(self, interior_radii: np.ndarray) -> float:
322         """
323         Calculer une pénalité pour les violations de contraintes.
324         PÉNALITÉS TRÈS FORTES pour forcer la forme hyperboloïde.
325         """
326         interior_radii = np.asarray(interior_radii)
327         # Accept either interior radii (length n_interior) or full radii (length
328             n_points)
329         if interior_radii.shape[0] == self.n_points:
330             radii = interior_radii.copy()
331         else:
332             radii = self.get_full_radii(interior_radii)
333         penalty = 0.0
334
335         # Pénalité monotonie depuis le bas ( $r[i]$  doit être  $< r[i-1]$ )
336         for idx in range(1, self.waist_idx + 1):
337             if radii[idx] >= radii[idx - 1]:
```

```
337         diff = radii[idx] - radii[idx - 1] + 0.1
338         penalty += diff ** 2 * 10000 # TRÈS FORT
339
340     # Pénalité monotonie depuis le haut (r[i] doit être < r[i+1])
341     for idx in range(self.waist_idx, self.n_points - 1):
342         if radii[idx] >= radii[idx + 1]:
343             diff = radii[idx] - radii[idx + 1] + 0.1
344             penalty += diff ** 2 * 10000 # TRÈS FORT
345
346     # Pénalité convexité bas (d2r > 0)
347     for idx in range(1, self.waist_idx + 1):
348         # centred second difference at idx
349         if idx >= 1 and idx <= self.n_points - 2:
350             d2r = radii[idx - 1] - 2 * radii[idx] + radii[idx + 1]
351             if d2r < 0:
352                 penalty += abs(d2r) ** 2 * 5000
353
354     # Pénalité convexité haut (d2r > 0)
355     for idx in range(self.waist_idx, self.n_points - 1):
356         if idx >= 1 and idx <= self.n_points - 2:
357             d2r = radii[idx - 1] - 2 * radii[idx] + radii[idx + 1]
358             if d2r < 0:
359                 penalty += abs(d2r) ** 2 * 5000
360
361     return penalty
362
363
364 # =====
365 # OPTIMIZER CLASSES
366 # =====
367
368 class BaseOptimizer:
369     """Base class for all optimization algorithms."""
370
371     def __init__(self, problem: CoolingTowerProblem, case_num: int = 1,
372                 geo_constraints: GeometricConstraints = None,
373                 fixed_radii: np.ndarray = None, **kwargs):
374         self.problem = problem
375         self.case_num = case_num
376         self.fixed_radii = fixed_radii
377         self.geo_constraints = geo_constraints
378         # Extract correction_mode before storing remaining kwargs as algo options
379         # 'correct' = correction-based (good for SA/BFGS)
380         # 'penalty' = penalty-based (smooth landscape, good for PSO)
381         self.correction_mode = kwargs.pop('correction_mode', 'correct')
382         self.options = kwargs
383
384     def _objective_func(self, x):
385         """Objective function with geometric constraints.
386
387         """
388         TwoModes(set_via_correction_mode):
```

```
388
389         'correct' (default, best for SA/BFGS):
390         1. Project the proposed solution to geometric feasibility
391         2. Evaluate cost on the corrected solution
392         3. Add distance penalty guiding optimizer toward feasible region
393
394         'penalty' (best for PSO):
395         1. Evaluate cost on the raw proposed solution
396         2. Add smooth geometric penalty for constraint violations
397         This gives PSO a smooth landscape without fixed-point attractors.
398
399         Cases 2, 6, 8 bypass geometric constraints in both modes.
400         """
401         # No geometric constraints → evaluate directly
402         if self.geo_constraints is None:
403             return self.problem.cost_function(
404                 x, self.case_num, fixed_radii=self.fixed_radii
405             )
406
407         # Case 2: only heights vary, radii are fixed → no geo correction needed
408         # Case 6: 1.5x volume target cannot be achieved with hyperboloid shape
409         #           (waist < rm=27.4 limits max volume), so skip geo constraints
410         # Case 8: hyperbola params [a,b,c], not radii → no geo correction needed
411         if self.case_num in [2, 6, 8]:
412             return self.problem.cost_function(
413                 x, self.case_num, fixed_radii=self.fixed_radii
414             )
415
416         # Extract interior radii from decision variables
417         try:
418             radii_full, heights = self.problem.unpack_decision_variables(
419                 x, self.case_num, fixed_radii=self.fixed_radii
420             )
421         except Exception:
422             return self.problem.cost_function(
423                 x, self.case_num, fixed_radii=self.fixed_radii
424             )
425
426         interior = np.asarray(radii_full)[1:-1]
427
428         if self.correction_mode == 'penalty':
429             # PENALTY MODE: smooth landscape for population-based methods (PSO)
430             # Evaluate on the raw (uncorrected) solution + add geo penalty
431             cost = self.problem.cost_function(
432                 x, self.case_num, fixed_radii=self.fixed_radii
433             )
434             geo_pen = self.geo_constraints.compute_penalty(interior)
435             return cost + geo_pen
436
437         # CORRECTION MODE: project-and-penalise for gradient-based / local methods
438         corrected = self.geo_constraints.correct_solution(interior, max_iterations=20)
```

```
439
440     # Reconstruct decision vector from corrected radii
441     if self.case_num in [1, 4, 5, 6, 7]:
442         x_eval = np.asarray(corrected)
443     elif self.case_num == 3:
444         x_eval = np.concatenate((np.asarray(corrected), np.asarray(heights)))
445     else:
446         x_eval = x
447
448     # Cost on the corrected (feasible) solution
449     cost = self.problem.cost_function(
450         x_eval, self.case_num, fixed_radii=self.fixed_radii
451     )
452
453     # Distance penalty: guides optimizer toward feasible solutions
454     dist = np.sum((interior - corrected) ** 2)
455     return cost + 1e4 * dist
456
457     def optimize(self, initial_guess: np.ndarray):
458         raise NotImplementedError
459
460
461 class SaOptimizer(BaseOptimizer):
462     """Simulated Annealing Wrapper."""
463
464     def optimize(self, initial_guess: np.ndarray):
465         start_time = time.time()
466
467         if simulated_annealing is None:
468             raise NotImplementedError("sa.py not found.")
469
470         bounds = self.problem.get_bounds(self.case_num)
471
472         best_x, best_cost, history, n_fevals = simulated_annealing(
473             objective_func=self._objective_func,
474             initial_guess=initial_guess,
475             bounds=bounds,
476             **self.options
477         )
478
479         # PAS de correction finale - les pénalités guident l'optimisation
480         # La solution doit naturellement satisfaire les contraintes
481
482         time_taken = time.time() - start_time
483         metrics = {"n_fevals": n_fevals, "time_taken": time_taken}
484         return best_x, best_cost, history, metrics
485
486
487 class BfgsOptimizer(BaseOptimizer):
488     """BFGS Wrapper."""
489
```

```
490     def optimize(self, initial_guess: np.ndarray):
491         start_time = time.time()
492
493         if bfgs_optimize is None:
494             raise NotImplementedError("bfgs.py not found.")
495
496         bounds = self.problem.get_bounds(self.case_num)
497
498         best_x, best_cost, history, n_fevals = bfgs_optimize(
499             objective_func=self._objective_func,
500             initial_guess=initial_guess,
501             bounds=bounds,
502             **self.options
503         )
504
505         # PAS de correction finale - les pénalités guident l'optimisation
506
507         time_taken = time.time() - start_time
508         metrics = {"n_fevals": n_fevals, "time_taken": time_taken}
509         return best_x, best_cost, history, metrics
510
511
512 class PsoOptimizer(BaseOptimizer):
513     """Particle Swarm Wrapper."""
514
515     def optimize(self, initial_guess: np.ndarray):
516         start_time = time.time()
517
518         if particle_swarm is None:
519             raise NotImplementedError("pso.py not found.")
520
521         bounds = self.problem.get_bounds(self.case_num)
522
523         best_x, best_cost, history, n_fevals = particle_swarm(
524             objective_func=self._objective_func,
525             bounds=bounds,
526             **self.options
527         )
528
529         # PAS de correction finale - les pénalités guident l'optimisation
530
531         time_taken = time.time() - start_time
532         metrics = {"n_fevals": n_fevals, "time_taken": time_taken}
533         return best_x, best_cost, history, metrics
534
535
536 # =====
537 # SCENARIO RUNNER
538 # =====
539
540 def run_scenario(optimizer_class, problem: CoolingTowerProblem, case_num: int,
```

```
541         scenario_name: str, fixed_radii: np.ndarray = None,
542         use_geo_constraints: bool = True, **extra_kwargs):
543     """Run a specific scenario with geometric constraints."""
544
545     print(f"\n{'='*60}")
546     print(f"Scenario_{case_num}:_{scenario_name}")
547     print(f"Algorithm:_{optimizer_class.__name__}")
548     print(f"Geometric Constraints:_{'ON' if use_geo_constraints else 'OFF'}")
549     print(f"{'='*60}")
550
551     # Create geometric constraints
552     geo_constraints = None
553     if use_geo_constraints:
554         geo_constraints = GeometricConstraints(
555             r0=problem.r0,
556             rm=problem.rm,
557             n_points=problem.m + 1
558         )
559
560     # If fixed_radii provided for cases like Case 2, ensure they respect geometry (
561       correct if needed)
562     if use_geo_constraints and fixed_radii is not None:
563         fr = np.asarray(fixed_radii)
564         # Accept either interior or full radii
565         if fr.shape[0] == problem.m + 1:
566             interior_fr = fr[1:-1]
567         else:
568             interior_fr = fr
569
570         report_fr = geo_constraints.check_all_points(interior_fr, verbose=False)
571         if not report_fr['all_valid']:
572             corrected = geo_constraints.correct_solution(interior_fr, max_iterations=50)
573             fixed_radii = np.concatenate(([geo_constraints.r0], corrected, [
574                 geo_constraints.rm]))
575
576     optimizer = optimizer_class(
577         problem,
578         case_num=case_num,
579         geo_constraints=geo_constraints,
580         fixed_radii=fixed_radii,
581         **extra_kwargs
582     )
583
584     initial_guess = problem.get_initial_guess(case_num)
585     init_radii, init_heights = problem.unpack_decision_variables(
586         initial_guess, case_num, fixed_radii=fixed_radii
587     )
588
589     # Run optimization
590     try:
591         best_x, best_cost, history, metrics = optimizer.optimize(initial_guess)
```

```
except NotImplementedError as e:
    print(f"Skipping: {e}")
    return None, None, None, None

best_radii, best_heights = problem.unpack_decision_variables(
    best_x, case_num, fixed_radii=fixed_radii
)

# Apply final geometric correction only for cases where interior radii are optimised
# Skip for Case 2 (heights only), Case 6 (needs wider shapes), Case 8 (hyperbola
params)
if geo_constraints is not None and case_num not in [2, 6, 8]:
    interior_best = np.asarray(best_radii)[1:-1]
    corrected_interior = geo_constraints.correct_solution(interior_best,
        max_iterations=50)
    best_radii = np.concatenate(([geo_constraints.r0], corrected_interior, [
        geo_constraints.rm]))
    report = geo_constraints.check_all_points(corrected_interior, verbose=True)
    print(f" {GeoCheck} Final: {'VALID' if report['all_valid'] else 'INVALID'}")

# Compute metrics on the FINAL (corrected) solution
from Problem.cooling_tower import total_surface_area, total_volume
final_area = total_surface_area(best_radii, best_heights)
final_vol = total_volume(best_radii, best_heights)

target_vol = problem.v_target * 1.5 if case_num == 6 else problem.v_target
vol_error = abs(final_vol - target_vol)

# True cost = surface area + volume penalty (NO geometric penalty inflating it)
# Uses the same penalty_weight as CoolingTowerProblem.cost_function
penalty_weight = 1e5
final_cost = final_area + penalty_weight * (final_vol - target_vol)**2

# == Save Results ==
import csv
import matplotlib.pyplot as plt

algo_name = optimizer_class.__name__.replace('Optimizer', '')
base_out_dir = os.path.join(os.path.dirname(__file__), "output")
out_dir = os.path.join(base_out_dir, algo_name)
os.makedirs(out_dir, exist_ok=True)

base_filename = f"Case{case_num}_{algo_name}"

# 1. Convergence Plot
plt.figure()
plt.plot(history, label='Cost')
plt.title(f"Convergence: Case {case_num} {algo_name}")
plt.xlabel("Iteration")
plt.ylabel("Cost J(x)")
plt.yscale('log')
```

```
plt.grid(True)
plt.legend()
plt.savefig(os.path.join(out_dir, f"{base_filename}_convergence.png"))
plt.close()

# 2. Profile Plot with Initial Guess
z_coords = np.concatenate(([0], np.cumsum(best_heights)))
z_coords_init = np.concatenate(([0], np.cumsum(init_heights)))

plt.figure(figsize=(8, 10))

# Initial Guess (ROUGE)
plt.plot(init_radii, z_coords_init, 'r--s', alpha=0.6, markersize=8, label='Initial_
    Guess')
plt.plot(-init_radii, z_coords_init, 'r--s', alpha=0.6, markersize=8)

# Optimized (BLEU)
plt.plot(best_radii, z_coords, 'b-o', markersize=6, label='Optimized')
plt.plot(-best_radii, z_coords, 'b-o', markersize=6)
plt.fill_betweenx(z_coords, -best_radii, best_radii, alpha=0.15, color='blue')

# Mark waist
waist_idx = len(best_radii) // 2
plt.plot([best_radii[waist_idx], -best_radii[waist_idx]],
        [z_coords[waist_idx], z_coords[waist_idx]],
        'g--', linewidth=2, label=f'Waist (r={best_radii[waist_idx]:.1f}m)')

plt.title(f"Profile: Case {case_num} ({algo_name})\nArea: {final_area:.2f} | Vol_
    Error: {vol_error:.2f}")
plt.xlabel("Radius (m)")
plt.ylabel("Height (m)")
plt.legend(loc='upper_right')
plt.grid(True)
plt.axis('equal')
plt.savefig(os.path.join(out_dir, f"{base_filename}_profile.png"), dpi=150)
plt.close()

# 3. Plot 3D Wireframe
fig = plt.figure(figsize=(8, 8))
ax = fig.add_subplot(111, projection='3d')
theta = np.linspace(0, 2*np.pi, 30)

# Create the 3D surface grid
for i in range(len(best_heights)):
    z_bot = z_coords[i]
    z_top = z_coords[i+1]
    r_bot = best_radii[i]
    r_top = best_radii[i+1]

# Simple frustum side
z_grid = np.linspace(z_bot, z_top, 5)
```

```
687     r_grid = np.linspace(r_bot, r_top, 5)
688
689     Z, Theta = np.meshgrid(z_grid, theta)
690     R, _ = np.meshgrid(r_grid, theta)
691
692     X = R * np.cos(Theta)
693     Y = R * np.sin(Theta)
694
695     ax.plot_wireframe(X, Y, Z, color='b', alpha=0.5)
696
697     ax.set_title(f"3D Wireframe: Case {case_num} ({algo_name})")
698     ax.set_xlabel('X(m)')
699     ax.set_ylabel('Y(m)')
700     ax.set_zlabel('Z(m)')
701
702     # Fix aspect ratio manually
703     max_radius = np.max(best_radii)
704     max_height = np.max(z_coords)
705     ax.set_xlim([-max_radius, max_radius])
706     ax.set_ylim([-max_radius, max_radius])
707     ax.set_zlim([0, max_height])
708     ax.set_box_aspect([1, 1, max_height / (2*max_radius)]) # Set 3D aspect ratio
709
710     wireframe_path = os.path.join(out_dir, f"{base_filename}_wireframe.png")
711     plt.savefig(wireframe_path)
712     plt.close()
713
714     # 3. Save Metrics to CSV
715     csv_path = os.path.join(base_out_dir, "summary_metrics.csv")
716     file_exists = os.path.isfile(csv_path)
717
718     with open(csv_path, mode='a', newline='') as f:
719         writer = csv.writer(f)
720         if not file_exists:
721             writer.writerow([
722                 "Scenario", "Algorithm", "TimeTaken_sec", "FunctionEvals",
723                 "FinalArea", "FinalVolume", "TargetVolume", "VolumeError",
724                 "FinalCost", "BestRadii", "BestHeights"
725             ])
726         writer.writerow([
727             scenario_name,
728             algo_name,
729             metrics.get('time_taken', 0.0),
730             metrics.get('n_fevals', 0),
731             final_area,
732             final_vol,
733             target_vol,
734             vol_error,
735             final_cost,
736             str(best_radii.tolist()),
737             str(best_heights.tolist())
```

```
738         ])
739
740     print(f"\nSaved artifacts to {out_dir}:")
741     print(f"_{base_filename}_convergence.png")
742     print(f"_{base_filename}_profile.png")
743     print(f"_{summary_metrics.csv}(Appended)")
744
745     print(f"\n---Results---")
746     print(f"Time: {metrics.get('time_taken', 0.0):.4f}s")
747     print(f"FuncEvals: {metrics.get('n_fevals', 0)}")
748     print(f"Final Area: {final_area:.2f}m2")
749     print(f"Final Vol: {final_vol:.2f}m3 (Target: {target_vol:.2f})")
750     print(f"Vol Error: {vol_error:.2f}m3")
751     print(f"Final Cost: {final_cost:.2f}")
752
753     return best_radii, best_heights, history, metrics
754
755
756
757 def generate_comparison_convergence(case_num, histories_dict, out_dir):
758     """
759     Plots convergence histories for all algorithms on a single figure for a given case.
760
761     Args:
762         case_num: The scenario number.
763         histories_dict: dict of {algo_name: history_list}.
764         out_dir: Output directory path.
765     """
766     import matplotlib.pyplot as plt
767
768     if not histories_dict:
769         return
770
771     plt.figure(figsize=(10, 6))
772     colors = {'Sa': 'red', 'Bfgs': 'blue', 'Pso': 'green'}
773     for algo_name, history in histories_dict.items():
774         if history is not None and len(history) > 1:
775             plt.plot(history, label=algo_name, color=colors.get(algo_name, 'black'))
776
777     plt.title(f"Convergence Comparison: Case {case_num}")
778     plt.xlabel("Iteration")
779     plt.ylabel("Cost J(x)")
780     plt.yscale('log')
781     plt.grid(True)
782     plt.legend()
783     path = os.path.join(out_dir, f"Case{case_num}_comparison_convergence.png")
784     plt.savefig(path)
785     plt.close()
786     print(f"Saved: {path}")
787
788
```

```
789 def generate_comparison_table(out_dir):
790     """
791     Reads summary_metrics.csv and generates a formatted comparison table as a PNG image.
792     """
793     import csv
794     import matplotlib.pyplot as plt
795
796     csv_path = os.path.join(out_dir, "summary_metrics.csv")
797     if not os.path.isfile(csv_path):
798         print("No summary_metrics.csv found.")
799         return
800
801     with open(csv_path, 'r') as f:
802         reader = csv.reader(f)
803         rows = list(reader)
804
805     if len(rows) < 2:
806         print("Not enough data for comparison table.")
807         return
808
809     header = rows[0]
810     data = rows[1:]
811
812     # Build table with key columns only
813     table_header = ["Scenario", "Algorithm", "Time(s)", "F.Evals", "FinalArea", "Vol_
Error", "FinalCost"]
814     table_data = []
815     for row in data:
816         table_data.append([
817             row[0],                                # Scenario
818             row[1],                                # Algorithm
819             f"{float(row[2]):.4f}",                 # TimeTaken_sec
820             row[3],                                # FunctionEvals
821             f"{float(row[4]):.2f}",                 # FinalArea
822             f"{float(row[7]):.2f}",                 # VolumeError
823             f"{float(row[8]):.2f}"                  # FinalCost
824         ])
825
826     fig, ax = plt.subplots(figsize=(16, max(4, len(table_data) * 0.4 + 1)))
827     ax.axis('off')
828     ax.set_title("Optimization Results Comparison Table", fontsize=14, fontweight='bold',
, pad=20)
829
830     table = ax.table(
831         cellText=table_data,
832         colLabels=table_header,
833         cellLoc='center',
834         loc='center'
835     )
836     table.auto_set_font_size(False)
837     table.set_fontsize(8)
```

```
838     table.scale(1, 1.3)
839
840     # Color the header row
841     for j in range(len(table_header)):
842         table[(0, j)].set_facecolor('#4472C4')
843         table[(0, j)].set_text_props(color='white', fontweight='bold')
844
845     # Alternate row shading
846     for i in range(1, len(table_data) + 1):
847         color = '#D9E2F3' if i % 2 == 0 else '#FFFFFF'
848         for j in range(len(table_header)):
849             table[(i, j)].set_facecolor(color)
850
851     table_path = os.path.join(out_dir, "comparison_table.png")
852     plt.savefig(table_path, bbox_inches='tight', dpi=150)
853     plt.close()
854     print(f"Saved comparison table: {table_path}")
855
856
857 if __name__ == "__main__":
858     import os
859
860     problem = CoolingTowerProblem()
861
862     # Create output directory and clear old CSV to avoid stale data
863     out_dir = os.path.join(os.path.dirname(__file__), "output")
864     os.makedirs(out_dir, exist_ok=True)
865     csv_path = os.path.join(out_dir, "summary_metrics.csv")
866     if os.path.isfile(csv_path):
867         os.remove(csv_path)
868
869     print("Running all 8 cases across all 3 algorithms...")
870
871     # List of all available optimizer classes
872     optimizers = [SaOptimizer, BfgsOptimizer, PsoOptimizer]
873
874     # Scenarios
875     scenarios = [
876         (1, "Mandatory_Ref_Case"),
877         (2, "Variable_Heights"),
878         (3, "Full_Freedom"),
879         (4, "Tight_Waist"),
880         (5, "Cylindrical_Start"),
881         (6, "High_Volume"),
882         (7, "Restricted_Bounds"),
883         (8, "Hyperbolic_Fit")
884     ]
885
886     # Collect convergence histories per case for comparison plots
887     # Structure: {case_num: {algo_name: history}}
888     all_histories = {case_num: {} for case_num, _ in scenarios}
```

```
889
890 for opt_class in optimizers:
891     print(f"\n\n{' '*60}")
892     print(f"***_Starting_Runs_for_{opt_class.__name__}_***")
893     print(f"{' '*60}\n")
894
895     algo_name = opt_class.__name__.replace('Optimizer', '')
896     case_1_best_radaii = None
897
898     # Configure algorithm parameters for good convergence
899     kwargs = {}
900     if algo_name == 'Pso':
901         kwargs = {
902             'correction_mode': 'correct', # smooth landscape for swarm
903         }
904     elif algo_name == 'Sa':
905         kwargs = {'T_initial': 10000, 'max_iter': 15000}
906
907     for case_num, scenario_name in scenarios:
908         if case_num == 2:
909             if case_1_best_radaii is None:
910                 print("Skipping_Case_2:_Requires_radaii_from_Case_1.")
911                 continue
912             best_radaii, best_heights, history, metrics = run_scenario(
913                 opt_class, problem, case_num, scenario_name, fixed_radaii=
914                     case_1_best_radaii, **kwargs
915             )
916         else:
917             best_radaii, best_heights, history, metrics = run_scenario(
918                 opt_class, problem, case_num, scenario_name, **kwargs
919             )
920
921         if case_num == 1 and best_radaii is not None:
922             case_1_best_radaii = best_radaii
923
924         # Store history for comparison convergence plots
925         all_histories[case_num][algo_name] = history
926
927     # Generate comparison convergence plots (all 3 algos on one figure per case)
928     print("\n\nGenerating_comparison_convergence_plots...")
929     for case_num, _ in scenarios:
930         generate_comparison_convergence(case_num, all_histories[case_num], out_dir)
931
932     # Generate comparison table
933     print("\n\nGenerating_comparison_table...")
934     generate_comparison_table(out_dir)
935
936     print("\n\nAll_runs_complete!_Check_the_'output'_folder_for_all_artifacts.")
```

Appendix A.C BFGS Implementation — bfgs.py

The BFGS quasi-Newton implementation, including inverse Hessian approximation updates, Armijo line search, and numerical gradient computation via finite differences.

Listing 3: BFGS Implementation (bfgs.py)

```
1 import numpy as np
2 from typing import Callable, Tuple, List, Optional, Sequence
3
4 def shape_guidance_penalty(
5     objective_func, x, smooth_weight=0.0, slope_weight=0.0,
6     slope_min=0.20, slope_max=10.0, active_cases=None
7 ):
8     """Fallback dummy function if true shapes guidance is missing."""
9     return 0.0
10
11
12 def bfgs_optimize(
13     objective_func: Callable[[np.ndarray], float],
14     initial_guess: np.ndarray,
15     **kwargs
16 ) -> Tuple[np.ndarray, float, List[float], int]:
17     """
18     Martin's BFGS implementation.
19
20     Args:
21         objective_func: The function to minimize. Takes an array of variables and
22             returns the cost.
23         initial_guess: The starting point for the algorithm.
24
25     Returns:
26         best_position (np.ndarray): The best combination of parameters found.
27         best_cost (float): The cost evaluated at the best position.
28         history (List[float]): The history of the best cost over iterations.
29         n_fevals (int): Total number of times the objective function was evaluated.
30
31     """
32     max_iter = int(kwargs.get("max_iter", 300))
33     tol_grad = float(kwargs.get("tol_grad", kwargs.get("gtol", 1e-6)))
34     tol_step = float(kwargs.get("tol_step", 1e-10))
35     fd_eps = float(kwargs.get("fd_eps", kwargs.get("epsilon", 1e-6)))
36     c1 = float(kwargs.get("c1", 1e-4))
37     backtrack_beta = float(kwargs.get("backtrack_beta", 0.5))
38     min_step = float(kwargs.get("min_step", 1e-12))
39     verbose = bool(kwargs.get("verbose", False))
40     reset_hessian_on_bad_curvature = bool(kwargs.get("reset_hessian_on_bad_curvature",
41                                                         True))
42     use_shape_guidance = bool(kwargs.get("use_shape_guidance", True))
43     # Stronger shape guidance defaults to discourage flat vertical stacks
44     # while keeping flexibility at the fixed-end transitions.
```

```
42 shape_smooth_weight = float(kwargs.get("shape_smooth_weight", 0.0))
43 shape_slope_weight = float(kwargs.get("shape_slope_weight", 5e4))
44 shape_slope_min = float(kwargs.get("shape_slope_min", 0.20))
45 shape_slope_max = float(kwargs.get("shape_slope_max", 10.0))
46 shape_active_cases = kwargs.get("shape_active_cases", (1, 2, 3, 4, 5, 6, 7))
47
48 x = np.asarray(initial_guess, dtype=float).reshape(-1)
49 n_vars = x.size
50
51 bounds = kwargs.get("bounds")
52 if bounds is None and kwargs.get("use_problem_bounds", True):
53     bounds = _infer_bounds_from_objective(objective_func)
54 lb, ub = _parse_bounds(bounds, n_vars)
55 has_bounds = lb is not None
56
57 if has_bounds:
58     x = _project_box(x, lb, ub)
59
60 n_fevals = 0
61 best_x = x.copy()
62 best_cost = np.inf
63
64 def evaluate(z: np.ndarray) -> float:
65     nonlocal n_fevals, best_cost, best_x
66     z = np.asarray(z, dtype=float)
67     if has_bounds:
68         z = _project_box(z, lb, ub)
69
70     cost = float(objective_func(z))
71     if use_shape_guidance:
72         cost += shape_guidance_penalty(
73             objective_func=objective_func,
74             x=z,
75             smooth_weight=shape_smooth_weight,
76             slope_weight=shape_slope_weight,
77             slope_min=shape_slope_min,
78             slope_max=shape_slope_max,
79             active_cases=shape_active_cases
80         )
81
82     if not np.isfinite(cost):
83         cost = np.finfo(float).max
84     n_fevals += 1
85     if cost < best_cost:
86         best_cost = cost
87         best_x = z.copy()
88     return cost
89
90 def numerical_gradient(z: np.ndarray) -> np.ndarray:
91     grad = np.zeros_like(z, dtype=float)
92     for i in range(n_vars):
```

```
93         step = fd_eps * max(1.0, abs(z[i]))
94         z_plus = z.copy()
95         z_minus = z.copy()
96         z_plus[i] += step
97         z_minus[i] -= step
98
99         if has_bounds:
100             z_plus = _project_box(z_plus, lb, ub)
101             z_minus = _project_box(z_minus, lb, ub)
102
103         denom = z_plus[i] - z_minus[i]
104         if abs(denom) < 1e-15:
105             grad[i] = 0.0
106             continue
107
108         f_plus = evaluate(z_plus)
109         f_minus = evaluate(z_minus)
110         grad[i] = (f_plus - f_minus) / denom
111
112     return grad
113
114 def armijo_backtracking(z: np.ndarray, p: np.ndarray, fz: float, g: np.ndarray):
115     alpha = 1.0
116     g_tp = float(np.dot(g, p))
117
118     # Ensure descent direction.
119     if g_tp >= 0.0:
120         p = -g
121         g_tp = -float(np.dot(g, g))
122
123     while alpha >= min_step:
124         z_new = z + alpha * p
125         if has_bounds:
126             z_new = _project_box(z_new, lb, ub)
127         f_new = evaluate(z_new)
128         if f_new <= fz + c1 * alpha * g_tp:
129             return alpha, z_new, f_new
130         alpha *= backtrack_beta
131
132     z_new = z + min_step * p
133     if has_bounds:
134         z_new = _project_box(z_new, lb, ub)
135     return min_step, z_new, evaluate(z_new)
136
137 f_x = evaluate(x)
138 g = numerical_gradient(x)
139 inv_hessian = np.eye(n_vars, dtype=float)
140
141 history: List[float] = [float(best_cost)]
142
143 for k in range(1, max_iter + 1):
```

```
144     grad_norm = float(np.linalg.norm(g))
145     if grad_norm < tol_grad:
146         break
147
148     if not np.all(np.isfinite(inv_hessian)) or np.linalg.norm(inv_hessian, ord=np.
149         inf) > 1e10:
150         inv_hessian = np.eye(n_vars, dtype=float)
151     direction = -inv_hessian @ g
152     alpha, x_new, f_new = armijo_backtracking(x, direction, f_x, g)
153     s = x_new - x
154
155     if float(np.linalg.norm(s)) < tol_step:
156         x, f_x = x_new, f_new
157         history.append(float(best_cost))
158         break
159
160     g_new = numerical_gradient(x_new)
161     y = g_new - g
162     y_s = float(np.dot(y, s))
163     curvature_scale = float(np.linalg.norm(y) * np.linalg.norm(s))
164     strong_curvature = y_s > max(1e-8, 1e-6 * curvature_scale)
165
166     if strong_curvature and np.isfinite(y_s):
167         rho = 1.0 / y_s
168         if not np.isfinite(rho) or rho > 1e10:
169             if reset_hessian_on_bad_curvature:
170                 inv_hessian = np.eye(n_vars, dtype=float)
171             else:
172                 eye = np.eye(n_vars, dtype=float)
173                 v = eye - rho * np.outer(s, y)
174                 try:
175                     with np.errstate(over="raise", divide="raise", invalid="raise"):
176                         h_candidate = v @ inv_hessian @ v.T + rho * np.outer(s, s)
177                         if np.all(np.isfinite(h_candidate)):
178                             inv_hessian = h_candidate
179                         elif reset_hessian_on_bad_curvature:
180                             inv_hessian = np.eye(n_vars, dtype=float)
181                     except FloatingPointError:
182                         if reset_hessian_on_bad_curvature:
183                             inv_hessian = np.eye(n_vars, dtype=float)
184                 elif reset_hessian_on_bad_curvature:
185                     inv_hessian = np.eye(n_vars, dtype=float)
186
187     if not np.all(np.isfinite(inv_hessian)) and reset_hessian_on_bad_curvature:
188         inv_hessian = np.eye(n_vars, dtype=float)
189
190     x, f_x, g = x_new, f_new, g_new
191     history.append(float(best_cost))
192
193     if verbose and (k == 1 or k % 25 == 0):
194         print(
```

```

194         f"[BFGS]_{iter}_{k:4d}/{max_iter}_{best_cost:_{best_cost:.6f}}|"
195         f"||g||:_{grad_norm:.3e}_{alpha:_{alpha:.3e}}"
196     )
197
198     return best_x.copy(), float(best_cost), history, int(n_fevals)
199
200
201 def _project_box(x: np.ndarray, lb: np.ndarray, ub: np.ndarray) -> np.ndarray:
202     return np.minimum(np.maximum(x, lb), ub)
203
204
205 def _parse_bounds(
206     bounds: Optional[Sequence[Tuple[Optional[float], Optional[float]]]],
207     n_vars: int
208 ) -> Tuple[Optional[np.ndarray], Optional[np.ndarray]]:
209     if bounds is None:
210         return None, None
211     if len(bounds) != n_vars:
212         raise ValueError(f"Expected_{n_vars}_bounds, got_{len(bounds)}")
213
214     lb = np.array(
215         [-np.inf if b[0] is None else float(b[0]) for b in bounds],
216         dtype=float
217     )
218     ub = np.array(
219         [np.inf if b[1] is None else float(b[1]) for b in bounds],
220         dtype=float
221     )
222     if np.any(lb > ub):
223         raise ValueError("Invalid_bounds: lower_bound_greater_than_upper_bound.")
224     return lb, ub
225
226
227 def _infer_bounds_from_objective(
228     objective_func: Callable[[np.ndarray], float]
229 ) -> Optional[Sequence[Tuple[float, float]]]:
230     """
231     Try to infer bounds from solver wrapper:
232     objective_func -> BaseOptimizer._objective_func -> {problem, case_num}.
233     """
234     owner = getattr(objective_func, "__self__", None)
235     if owner is None:
236         return None
237
238     problem = getattr(owner, "problem", None)
239     case_num = getattr(owner, "case_num", None)
240     if problem is None or case_num is None or not hasattr(problem, "get_bounds"):
241         return None
242
243     try:
244         return problem.get_bounds(case_num)

```

```
245     except Exception:  
246         return None
```

Appendix A.D PSO Implementation — pso.py

The Particle Swarm Optimisation implementation, including swarm initialisation, velocity updates, boundary handling, and convergence tracking.

Listing 4: Particle Swarm Optimisation (pso.py)

```
1 """
2 Subject: PSO algorithm
3 Inheritance: Particle_Swarm_Optimization.py from Task 3
4 @author: Cl  mence-Philom  ne HINOT, John HOARAU, Copilot AI
5 @date: 10/02/2026
6 """
7
8 # ***** IMPORTATION *****
9
10 import numpy as np
11 from typing import Callable, Tuple, List
12
13 # pymoo engine -- mirrors Particle_Swarm_Optimization.py
14 from pymoo.algorithms.soo.nonconvex.pso import PSO
15 from pymoo.core.problem import Problem
16 from pymoo.core.callback import Callback
17 from pymoo.core.termination import Termination
18 from pymoo.operators.sampling.lhs import LHS
19 from pymoo.optimize import minimize
20
21 # ***** CONVERGENCE CALLBACK *****
22
23 class MyCallback(Callback):
24     """
25     Tracks the global best cost after each generation.
26     Mirrors MyCallback in Particle_Swarm_Optimization_multi_obj.py.
27
28     data["best_f1"]: global best cost per generation, returned as history
29     solver.py can plot the convergence curve.
30     data["F"]: all particle costs per generation.
31     data["n_nds"]: number of particles that matched the global best.
32     """
33
34     def __init__(self):
35         super().__init__()
36         self.data["best_f1"] = []
37         self.data["F"] = []
38         self.data["n_nds"] = []
39
40     def notify(self, algorithm):
41         F = algorithm.pop.get("F") # shape (pop_size, 1)
42         best = float(F.min())
43         self.data["F"].append(F.copy())
```

```
44         self.data["best_f1"].append(best)
45         self.data["n_nds"].append(int((F == best).sum()))
46
47     # ***** PYMOO PROBLEM WRAPPER *****
48
49     class _FuncProblem(Problem):
50         """
51         Wraps a plain callable f(x) -> float into a pymoo Problem so that
52         pymoo's PSO engine can be used without changing solver.py's interface.
53         """
54
55         def __init__(self, objective_func: Callable, bounds: list):
56             lb = np.array([b[0] for b in bounds], dtype=float)
57             ub = np.array([b[1] for b in bounds], dtype=float)
58             super().__init__(
59                 n_var=len(bounds),
60                 n_obj=1,
61                 xl=lb,
62                 xu=ub,
63             )
64             self._func = objective_func
65             self.n_fevals = 0
66
67         def _evaluate(self, X, out, *args, **kwargs):
68             """Evaluate all particles in the current generation."""
69             F = np.array([self._func(X[i]) for i in range(len(X))], dtype=float)
70             self.n_fevals += len(X)
71             out["F"] = F.reshape(-1, 1)
72
73     # ***** STAGNATION-AWARE TERMINATION *****
74
75     class StagnationTermination(Termination):
76         """
77         Stops when EITHER:
78         - max_gen generations are reached, OR
79         - the global best has not improved by more than 'tol' over the
80         last 'stagnation_window' generations (early stopping).
81         """
82
83         def __init__(self, max_gen: int, stagnation_window: int = 50, tol: float = 1e-6):
84             super().__init__()
85             self.max_gen = max_gen
86             self.stagnation_window = stagnation_window
87             self.tol = tol
88             self._history = []
89
90         def _update(self, algorithm):
91             F = algorithm.pop.get("F")
92             self._history.append(float(F.min()))
93
94             if algorithm.n_gen >= self.max_gen:
```

```
95         return 1.0
96
97     if len(self._history) >= self.stagnation_window:
98         window = self._history[-self.stagnation_window:]
99         improvement = window[0] - window[-1]
100         if improvement < self.tol:
101             return 1.0
102
103     return algorithm.n_gen / self.max_gen
104
105 # ***** PARTICLE SWARM OPTIMISATION *****
106
107 """
108 Interface expected by solver.py (PsoOptimizer):
109
110 best_x, best_cost, history, n_fevals = particle_swarm(
111     objective_func=self._objective_func,
112     bounds=bounds,
113     num_particles=200,
114     max_iter=750,
115     ...
116 )
117
118 history is returned as an empty list, only the final result is needed.
119
120 Parameters
121 -----
122 objective_func: Callable[[x: np.ndarray] -> float], function to minimise.
123 bounds: list of (min, max) tuples, one tuple per decision variable.
124 num_particles: int, swarm size.
125 max_iter: int, number of generations.
126 w: float, inertia weight (pymoo adaptive=True decays this).
127 c1: float, cognitive coefficient.
128 c2: float, social coefficient.
129 max_velocity_rate: float, v_max = max_velocity_rate * (ub - lb).
130 seed: int | None, random seed for reproducibility.
131 verbose: bool, print pymoo progress each generation.
132 **kwargs: ignored, absorbs extra keys from solver.py.
133
134 Returns
135 -----
136 best_position: np.ndarray, best parameter vector found.
137 best_cost: float, objective value at best position.
138 history: [], empty list (history not tracked).
139 n_fevals: int, total objective-function evaluations.
140 """
141
142 def particle_swarm(
143     objective_func: Callable[[np.ndarray], float],
144     bounds: List[Tuple[float, float]],
145     num_particles: int = 200,
```

```
146     max_iter: int = 50000,
147     stagnation_window: int = 50,
148     tol: float = 1e-6,
149     w: float = 0.9,
150     c1: float = 2.0,
151     c2: float = 2.0,
152     max_velocity_rate: float = 0.3,
153     seed: int = 10,
154     verbose: bool = False,
155     **kwargs,
156 ) -> Tuple[np.ndarray, float, List[float], int]:
157
158     try:
159         # ----- Wrap objective into a pymoo Problem -----
160         problem = _FuncProblem(objective_func, bounds)
161
162         # ----- Configure PSO -- mirrors PSO_single_objective() -----
163         algorithm = PSO(
164             pop_size          = num_particles,
165             w                 = w,
166             c1                = c1,
167             c2                = c2,
168             max_velocity_rate = max_velocity_rate,
169             adaptive          = True,
170             initial_velocity  = "random",
171             pertube_best      = True,
172         )
173
174         # ----- Convergence callback -----
175         callback = MyCallback()
176
177         # ----- Stagnation-aware termination -----
178         termination = StagnationTermination(
179             max_gen          = max_iter,
180             stagnation_window = stagnation_window,
181             tol              = tol,
182         )
183
184         # ----- Run optimisation -----
185         result = minimize(
186             problem,
187             algorithm,
188             termination = termination,
189             seed        = seed,
190             verbose      = verbose,
191             callback     = callback,
192         )
193
194         # Store callback on result for external access
195         result.callback = callback
196
```

```
197     history = callback.data["best_f1"]  # one float per generation
198
199     return result.X.flatten(), float(result.F.flatten()[0]), history, problem.
        n_fevals
200
201 except Exception as e:
202     print(f"Error during PSO optimisation: {e}")
203     raise
```

Appendix A.E SA Implementation — sa.py

The Simulated Annealing implementation, including cooling schedule, neighbour generation, and Metropolis acceptance criterion.

Listing 5: Simulated Annealing (sa.py)

```
1 import numpy as np
2 from typing import Callable, Tuple, List
3
4
5
6 def simulated_annealing(
7     objective_func: Callable[[np.ndarray], float],
8     initial_guess: np.ndarray,
9     **kwargs
10 ) -> Tuple[np.ndarray, float, List[float], int]:
11     """
12     Pierre's Simulated Annealing implementation.
13
14     Args:
15         objective_func: The function to minimize. Takes an array of variables and
16             returns the cost.
17         initial_guess: The starting point for the algorithm.
18
19     Returns:
20         best_position (np.ndarray): The best combination of parameters found.
21         best_cost (float): The cost evaluated at the best position.
22         history (List[float]): The history of the best cost over iterations.
23         n_fevals (int): Total number of times the objective function was evaluated.
24     """
25
26     # ----- Extract parameters from kwargs -----
27     bounds = kwargs.get('bounds', None)
28     T_initial = kwargs.get('T_initial', 5000.0)
29     T_final = kwargs.get('T_final', 1e-8)
30     alpha = kwargs.get('alpha', 0.995)
31     max_iter = kwargs.get('max_iter', 5000)
32     n_neighbors = kwargs.get('n_neighbors', 3)
33     step_size = kwargs.get('step_size', 0.15)
34     adaptive_step = kwargs.get('adaptive_step', True)
35     seed = kwargs.get('seed', None)
36     verbose = kwargs.get('verbose', False)
37
38     if seed is not None:
39         np.random.seed(seed)
40
41     x = np.array(initial_guess, dtype=float)
42     n_vars = len(x)
```

```
43     # ----- Bounds handling -----
44     if bounds is not None:
45         lb = np.array([b[0] for b in bounds], dtype=float)
46         ub = np.array([b[1] for b in bounds], dtype=float)
47         rng = ub - lb # search-space width per dimension
48     else:
49         lb = None
50         ub = None
51         rng = np.abs(x) + 1.0 # fallback scaling
52
53     # ----- Initial evaluation -----
54     current_cost = objective_func(x)
55     n_fevals = 1
56
57     best_pos = x.copy()
58     best_cost = current_cost
59
60     history: List[float] = [best_cost]
61     log_every = max(1, max_iter // 10)
62
63     # ----- Temperature schedule -----
64     T = T_initial
65
66     # ----- Main optimisation loop -----
67     for iteration in range(max_iter):
68
69         # -- Adaptive step size: decreases with temperature --
70         # Allows large exploration at high T, fine-tuning at low T
71         if adaptive_step:
72             t_ratio = np.sqrt(T / T_initial)
73             t_ratio = np.clip(t_ratio, 0.01, 1.0)
74             step_current = step_size * t_ratio
75         else:
76             step_current = step_size
77
78         # -- Generate and evaluate neighbors --
79         # Sample n_neighbors candidates, keep the best for acceptance test
80         best_neighbor_x = None
81         best_neighbor_cost = np.inf
82
83         for _ in range(n_neighbors):
84             # Gaussian perturbation scaled by search range
85             perturbation = np.random.randn(n_vars) * step_current * rng
86             neighbor_x = x + perturbation
87
88             # Clipping boundary: project back onto bounds
89             if lb is not None and ub is not None:
90                 neighbor_x = np.clip(neighbor_x, lb, ub)
91
92             neighbor_cost = objective_func(neighbor_x)
93             n_fevals += 1
```

```
94
95         if neighbor_cost < best_neighbor_cost:
96             best_neighbor_cost = neighbor_cost
97             best_neighbor_x = neighbor_x.copy()
98
99     # -- Metropolis acceptance criterion --
100    delta_E = best_neighbor_cost - current_cost
101
102    if delta_E < 0:
103        # Better solution: always accept
104        accept = True
105    else:
106        # Worse solution: accept with probability exp(-ΔE/T)
107        acceptance_prob = np.exp(-delta_E / T)
108        accept = np.random.rand() < acceptance_prob
109
110    if accept and best_neighbor_x is not None:
111        x = best_neighbor_x
112        current_cost = best_neighbor_cost
113
114        # Update global best
115        if current_cost < best_cost:
116            best_cost = current_cost
117            best_pos = x.copy()
118
119    history.append(best_cost)
120
121    # -- Cooling: geometric decay --
122    T = alpha * T
123
124    # -- Early stopping --
125    if T < T_final:
126        if verbose:
127            print(f"[SA] Early stop at iter_{iteration+1}: T={T:.2e} < T_final={T_final:.2e}")
128        break
129
130    # -- Verbose output --
131    if verbose and (iteration + 1) % log_every == 0:
132        print(f"[SA] Iter_{iteration+1}>5d/{max_iter}|"
133              f"T:{T:.2e}| Best cost:{best_cost:.6f}")
134
135    return best_pos, best_cost, history, n_fevals
```

Appendix B Detailed Results Data

Appendix B.A BFGS Results

Table 17: BFGS optimisation results across all eight study cases.

Case	Time (s)	Func Evals	Area (m ²)	Vol (m ³)	Vol Error (m ³)
1 — Ref Case	0.1792	6 432	7 768.72	70 320.00	0.00
2 — Variable Heights	0.2042	6 432	7 776.30	70 320.00	0.00
3 — Full Freedom	0.3750	6 432	7 775.10	70 320.00	0.00
4 — Tight Waist	0.4962	6 432	7 775.11	70 320.00	0.00
5 — Cylindrical Start	1.4698	8 539	7 792.74	70 320.00	0.00
6 — High Volume	0.2520	6 426	7 678.24	105 480.00	0.00
7 — Restricted Bounds	0.1330	1 303	8 045.87	70 320.00	0.00
8 — Hyperbolic Fit	0.0146	294	5 961.77	70 320.00	0.00

Appendix B.B PSO Results

Table 18: PSO optimisation results across all eight study cases.

Case	Time (s)	Func Evals	Area (m ²)	Vol (m ³)	Vol Error (m ³)
1 — Ref Case	6.6602	150 200	8 021.28	33 740.58	36 579.42
2 — Variable Heights	4.2376	150 200	11 458.07	70 320.03	0.03
3 — Full Freedom	7.3397	150 200	9 908.86	55 449.77	14 870.23
4 — Tight Waist	6.8575	150 200	8 023.55	32 719.38	37 600.62
5 — Cylindrical Start	6.7781	150 200	8 072.50	37 934.59	32 385.41
6 — High Volume	4.1121	150 200	39 393.50	105 479.87	0.13
7 — Restricted Bounds	6.5794	150 200	8 041.84	43 941.57	26 378.43
8 — Hyperbolic Fit	5.7374	150 200	5 742.55	70 319.97	0.03

Appendix B.C SA Results

Results for the Simulated Annealing algorithm will be inserted once the SA solver runs have been completed and validated.